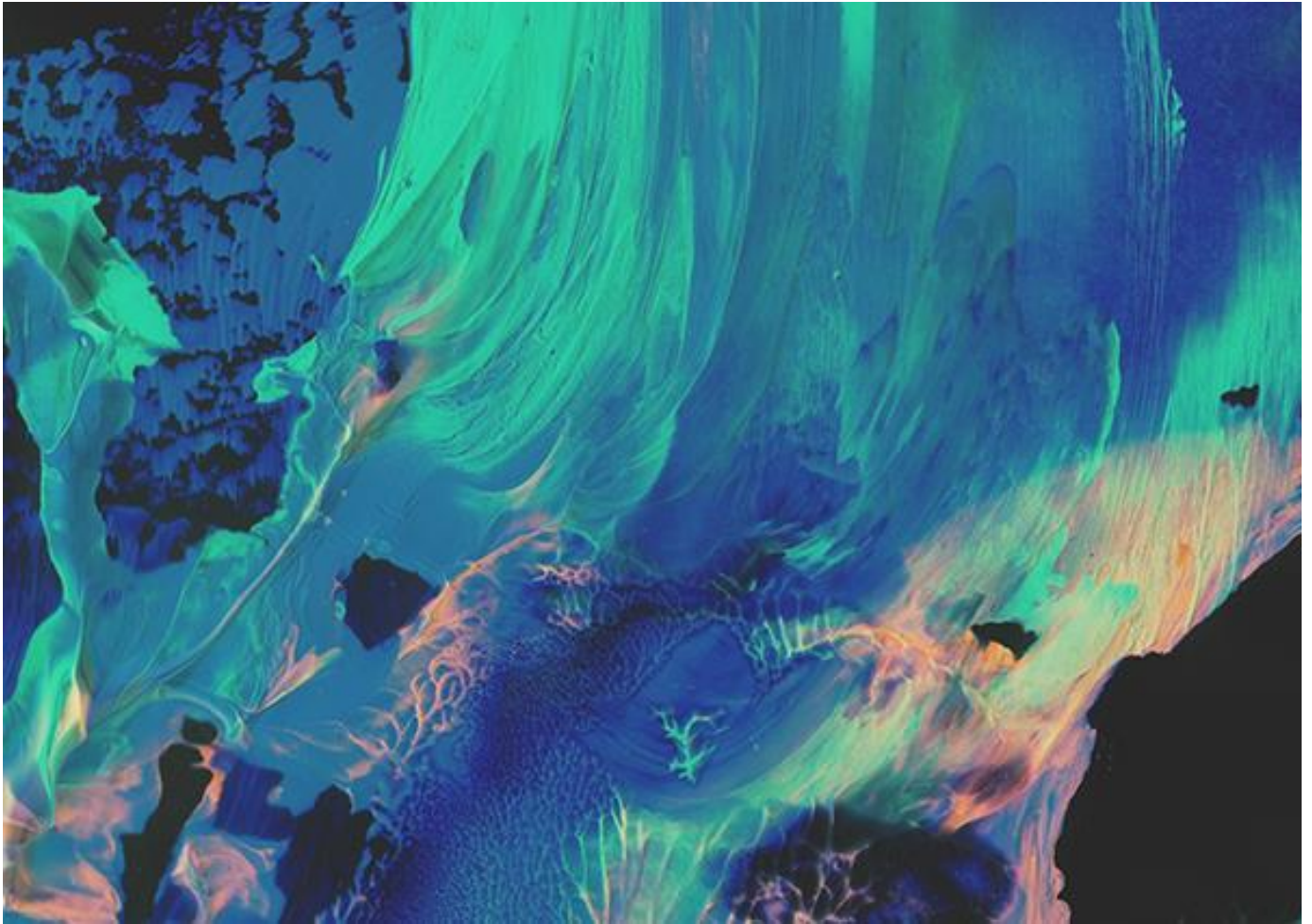




UMEÅ UNIVERSITY



Deep Reinforcement Learning for Option Hedging

Learning Adaptive Hedging Strategies under Transaction Costs

Felicia Aronsson
Jelena Nääs

Master thesis, 30 hp

M. Sc. Industrial Engineering & Management, 300 hp

Department of Mathematics and Mathematical Statistics

Spring term 2026

Deep Reinforcement Learning for Option Hedging

Learning Adaptive Hedging Strategies under Transaction Costs

Felicia Aronsson[†], `fear0017@student.umu.se`

Jelena Nääs[†], `anje0101@student.umu.se`

[†]*Equal contribution.*

COPYRIGHT © 2026 FELICIA ARONSSON & JELENA NÄÄS

ALL RIGHTS RESERVED

Supervisors: **Kevin Kamm** Umeå University
Victor Jonsson Vermiculus

Examiner: **Natalya Pya Arnqvist** Umeå University

Master Thesis, 30 hp

M.Sc. Industrial Engineering & Management, 300 hp

Department of Mathematics and Statistics

Umeå University

SE-901 87 Umeå, Sweden

Abstract

This thesis investigates the application of deep reinforcement learning (DRL) for option hedging in markets with transaction costs and discrete trading, and compares its performance with the traditional Black-Scholes-Merton (BSM) delta hedging approach. Although the BSM model provides a theoretically perfect hedge under idealized frictionless conditions, its performance degrades in real-world settings where constant re-balancing incurs costs. To address these limitations, four DRL agents-Deep Q-Network (DQN), Deep Deterministic Policy Gradient (DDPG), Twin Delayed Deep Deterministic Policy Gradient (TD3), and a proposed hierarchical Hybrid DQN-TD3-model were developed and evaluated. The agents were trained and tested across three market scenarios: a frictionless baseline, a realistic market (1% transaction costs), and a stressed market (5% transaction costs). The results demonstrate that all DRL agents converged to the theoretical BSM delta hedging strategy in the absence of frictions. In environments with transaction costs, the DRL agents outperformed the BSM benchmark in terms of mean average costs. These findings suggest that DRL agents can learn adaptive hedging strategies in non perfect markets.

Keywords: *Deep Reinforcement Learning, Option Hedging, Black-Scholes-Merton, Transaction Costs, DQN, DDPG, TD3, Hybrid Agents*

Sammanfattning

Denna avhandling undersöker tillämpningen av djup förstärkningsinlärning för optionshedging på marknader med transaktionskostnader och diskret handel, och jämför dess prestanda med den traditionella Black-Scholes-Merton (BSM) deltahedgingstrategin. Även om BSM-modellen ger en teoretiskt perfekt hedge under idealiserade friktionsfria förhållanden, försämras dess prestanda i verkliga marknader där konstant ombalansering medför höga kostnader. För att hantera dessa begränsningar utvecklades och utvärderades fyra djupa förstärkningsinlärningsagenter: Deep Q-Network (DQN), Deep Deterministic Policy Gradient (DDPG), Twin Delayed Deep Deterministic Policy Gradient (TD3) samt en föreslagen hierarkisk Hybrid DQN-TD3-modell. Agenterna tränades och testades i tre olika marknadsscenarier: en friktionsfri baslinjemiljö, en realistisk marknad med 1% transaktionskostnader samt en stressad marknad med 5% transaktionskostnader. Resultaten visar att samtliga agenter konvergerade mot den teoretiska Black-Scholes deltahedgingstrategin i frånvaro av marknadsfriktioner. I miljöer med transaktionskostnader överträffade agenterna Black-Scholes modellen i termer av genomsnittlig hedgingkostnad. Dessa resultat tyder på att agenter kan lära sig adaptiva hedgingstrategier i icke-perfekta marknader.

Acknowledgments

We would like to start by thanking our supervisor, Kevin Kamm at Umeå University, for his dedication, thoughtful guidance, and genuine enthusiasm throughout this project. His deep expertise and willingness to challenge us have been invaluable in shaping this thesis, and his belief in our ability to push further than we thought possible has meant a great deal to us.

We are also deeply grateful to Victor Jonsson and Vermiculus for the opportunity to carry out this thesis in collaboration with them. Their warm welcome, openness to our ideas and practical support have been highly appreciated.

Finally, we would like to thank our families, classmates, and friends for their support and companionship throughout our five years at Umeå University.

Felicia Aronsson & Jelena Nääs

Glossary

π The strategy that the agent will follow.

ATM At the money

BSM Black Scholes Merton model.

DDPG Deep Deterministic Policy Gradient.

DP Dynamic programming

DQN Deep Q-Network.

DRL Deep Reinforcement Learning

GBM Geometric Brownian Motion

Hybrid Hybrid agent with a combination of DQN and TD3.

ITM In the money

MC Monte Carlo

MDP Markov Decision Process

OTM Out of the money

OU Ornstein-Uhlenbeck noise

P&L Profit and Loss

RL Reinforcement Learning

RMSE Root Mean Squared Error

TD Temporal difference

TD3 Twin Delayed Deep Deterministic Policy Gradient.

TTM Time to Maturity

Contents

1. Introduction	1
1.1. Background	1
1.2. Literature Review	3
1.3. Aim of Thesis	4
2. Theory	5
2.1. Financial Background	5
2.1.1. Options	5
2.1.2. Hedging	6
2.1.3. The Black-Scholes-Merton Model	7
2.2. Reinforcement Learning Framework	8
2.2.1. Elements of Reinforcement Learning	8
2.2.2. Policy	9
2.3. Markov Decision Process	9
2.3.1. The MDP Tuple (S, A, P, R)	10
2.3.2. Markov Property	10
2.4. Return and Value Functions	11
2.4.1. Return and Reward Signal	11
2.4.2. Value Functions	12
2.5. The Bellman Equation	13
2.5.1. Bellman Optimality	14
2.6. Estimation of the Bellman Equations	16
2.6.1. Dynamic Programming	16
2.6.2. Monte Carlo Methods	17
2.6.3. Temporal Difference Learning	18
2.6.4. Q-learning	19
3. Method	21
3.1. Environment	21
3.1.1. Environment Definition	21
3.1.2. Environment Evolution	22
3.2. Agents	22
3.2.1. Deep Q-Network (DQN)	23
3.2.2. Deep Deterministic Policy Gradient (DDPG)	25
3.2.3. Twin Delayed Deep Deterministic Policy Gradient (TD3)	27
3.2.4. Hybrid Agent	28
3.3. Evaluation	30
4. Results and Discussion	32
4.1. Scenario A: The Frictionless Benchmark (No Transaction Costs)	33
4.1.1. Cost under a Frictionless Market	33

4.1.2.	Policy under a Frictionless Market	34
4.1.3.	Model Evaluation under Frictionless Markets	36
4.2.	Scenario B: The Realistic Market (Baseline Costs)	38
4.2.1.	Cost under Realistic market	38
4.2.2.	Policy under the Realistic Market	39
4.2.3.	Model Evaluation under the Realistic Market	40
4.3.	Scenario C: The Stress Test (High Transaction Costs)	42
4.3.1.	Cost for the Stress test	42
4.3.2.	Policy under the Stress Test	43
4.3.3.	Model Evaluation Under the Stressed Markets	45
4.4.	Discussion Around the Different Scenarios	45
4.5.	Hedging Trajectory	48
5.	Conclusion	50
5.1.	Summary	50
5.2.	Future Research	51
	References	52
A.	Parameters	53
A.1.	Discussion around the Hyperparameters	53
B.	Training the Agents for 100 and 1000 Episodes	55
C.	Hybrid Agent; Comparing Two Different Approaches	57
D.	Additional Results From the Stress Test	58
E.	Code	60

1. Introduction

Section 1.1 provides an overview of Reinforcement Learning and its applications, and Section 1.2 offers a brief overview of the literature and history of the topic. Section 1.3 outlines the purpose of the thesis together with the research questions.

1.1. Background

Have you ever wondered if we could mathematically define how leaning works? Intelligence isn't about memorizing facts; it's about the ability to adapt, make mistakes, and understand that the actions you take today affect the world that you encounter tomorrow. Reinforcement learning is the translation of this biological intuition into equations, allowing a machine to look into an uncertain future, navigate through trial and error, and discover optimal strategies for complex problems.

Machine Learning is a field defined by the specific types of problems it tackles and the computational methods created to solve them. Within this discipline, Reinforcement Learning (RL) is recognized as the third major machine learning paradigm, distinguishing itself from its counterparts: supervised and unsupervised learning. While supervised learning focuses on mapping inputs to outputs using explicitly labeled data for classification or regression,¹ and unsupervised learning seeks to uncover hidden structures and patterns within unlabeled data,² reinforcement learning operates on an entirely different basis. It is centered on learning how to make optimal decisions through active experience. Rather than learning from static, labeled datasets or discovering geometric structures in unlabeled data, an autonomous agent interacts with a dynamic environment by taking sequential actions and receiving feedback.

The philosophy of reinforcement learning is rooted in the intuitive concept that we naturally learn by interacting with our surroundings. Instead of relying on a pre-collected dataset or continuous external guidance, the agent learns to make decisions through iterative trial-and-error as it interacts dynamically with its environment.³ Under this framework, reinforcement learning problems are formalized by the challenge of learning how to map situational contexts to specific actions in order to maximize a cumulative numerical reward signal over time.

In practice, this process involves an agent, which can range from a software program to a physical robot or an algorithmic trading model, to navigate a defined environment. This environment could manifest as a physical space, a virtual simulation, or a financial market. As the agent selects and executes actions within this domain, its choices generate outcomes, some of which are more desirable than others.⁴ Consequently, RL can be applied to any system where sequential decision-making and control over time are essential. This sequen-

¹See <https://www.ibm.com/think/topics/supervised-learning>. Last accessed 25-05-26.

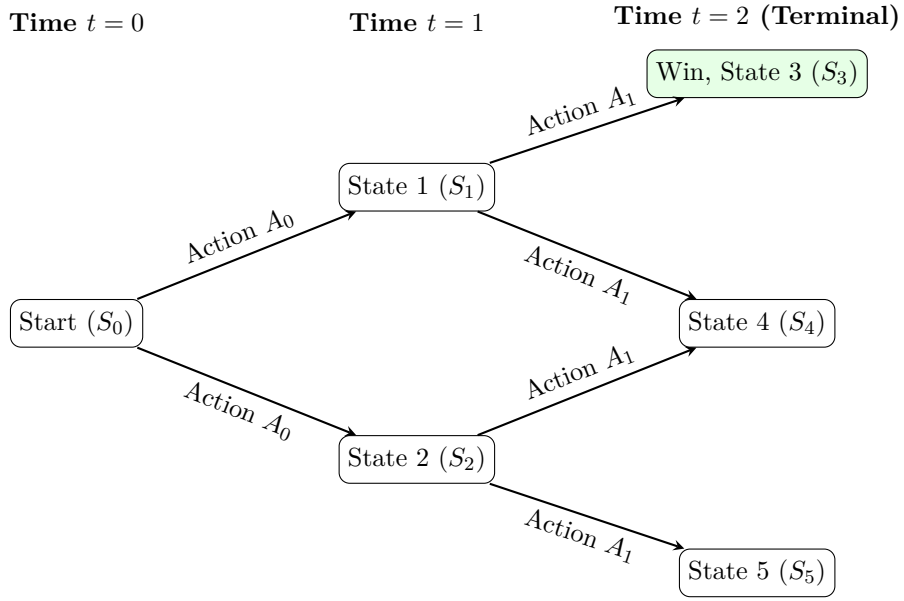
²See <https://www.mathworks.com/discovery/unsupervised-learning.html>. Last accessed 25.05.26.

³See <https://www.ibm.com/think/topics/reinforcement-learning>. Last accessed 25.05.26.

⁴See <https://cloud.google.com/discover/what-is-reinforcement-learning>. Last accessed 25.05.26.

tial trial-and-error approach makes reinforcement learning particularly suitable for complex problems in which decisions must be made dynamically under uncertainty and the optimal strategy cannot be easily derived analytically.

To illustrate how a reinforcement learning agent interacts with a dynamic environment, we can model a simple two-step binomial decision tree. Figure 1 visualizes the decision process for an agent operating within a binary action space, where it chooses to move either up or down at each juncture.



Agent receives reward R_t at time $t = 1, 2$

Figure 1: A simple two-step decision tree representing an agent navigating an environment through sequential choices.

The agent must select an action at each discrete time step t until it reaches a terminal state, at which point the final feedback is evaluated. Through repetitive episodes, the agent explores different pathways until it successfully learns the optimal policy. Through this continuous feedback loop of actions, state transitions, and subsequent rewards, the agent systematically optimizes its decision-making strategy.

In recent years, reinforcement learning has shown remarkable success in various fields. A well-known example is the work by MNIH et al. (2013), where RL combined with deep neural networks allowed algorithms to learn how to play six different Atari video games. The biggest breakthrough occurred when SILVER et al. (2016) demonstrated an algorithm that learned to play the game Go, which is considered one of the most challenging games for an AI to master. The agent beat the European Go champion 5 games to 0 and achieved a 99.8% winning rate

against other Go programs. These successes have boosted interest in applying reinforcement learning to other fields involving complex sequential decisions, including finance.

1.2. Literature Review

The theoretical foundations of reinforcement learning trace back to the 1950s, but the field did not attract much attention until MNH et al. (2013) demonstrated that a deep neural network could learn to play Atari video games at human-level performance without any prior knowledge of the rules. This work introduced the Deep Q-Network (DQN), which combined Q-learning with deep neural networks. Later research addressed key limitations of DQN, particularly its restriction to discrete action spaces. LILICRAP et al. (2015) introduced the Deep Deterministic Policy Gradient (DDPG) algorithm, which extends deep Q-learning to continuous action spaces using an actor-critic architecture, and FUJIMOTO et al. (2018) further improved this with the Twin Delayed Deep Deterministic Policy Gradient (TD3) algorithm, addressing overestimation bias and training instability. The hybrid architecture explored in this thesis is inspired by HE et al. (2025), which demonstrated that combining a DQN for high-level decision-making with a TD3 agent for low-level control improved performance in autonomous navigation tasks. The primary book on reinforcement learning theory used in this thesis is *Reinforcement Learning: An Introduction* by SUTTON and BARTO (2018).

The application of reinforcement learning to financial markets has grown in parallel with these algorithmic advances. The problem most directly related to this thesis is option hedging, where the traditional Black–Scholes delta-hedging strategy relies on idealized assumptions that fail in real markets. CAO et al. (2020) addressed this by applying deep reinforcement learning to derive optimal hedging strategies for a short call position in the presence of transaction costs, showing that reinforcement learning agents consistently perform better than the classical delta hedging in terms of mean hedging cost.

Use of Generative Artificial Intelligence

Generative artificial intelligence (AI) tools, including ChatGPT and Gemini, were used to improve the clarity, language, and overall flow of the thesis. This involved providing draft text to the AI systems, which were used to suggest revisions and improve readability. All content was reviewed and edited by the authors to ensure accuracy and consistency.

In addition, the AI tool Claude was used to support technical aspects of the project, including debugging code, generating plots, and creating an interactive HTML report to visualize and compare results across training runs.

1.3. Aim of Thesis

This thesis investigates the application of deep reinforcement learning (DRL) for option hedging in non-perfect markets. Financial markets present a natural setting for reinforcement learning, where there is no single correct action. Instead, the optimal choice depends on the full context of the situation and its consequences over time. This structure, in which an agent makes a sequence of interdependent decisions under uncertainty to maximize a long-term objective, is exactly what reinforcement learning is designed to handle.

A central challenge in financial risk management is accurately pricing and hedging financial derivatives. A widely used method for determining the hedge ratio for an option contract is the Black-Scholes-Merton model (BSM)⁵. While the BSM model provides a theoretically perfect hedge under idealized conditions, it relies on simplifying assumptions such as continuous trading and the absence of transaction costs, which are assumptions that do not hold in real markets.

To address these limitations, this thesis implements and evaluates four DRL agents: DQN, DDPG, TD3, and a Hybrid agent combining DQN and TD3. These agents compare their hedging performance against the classical BSM delta hedging benchmark across three market scenarios of increasing friction. This leads to the following research questions:

- RQ1** Do deep reinforcement learning agents converge to the Black-Scholes-Merton delta hedging strategy when trained in a frictionless market environment?
- RQ2** Can deep reinforcement learning agents learn hedging strategies that perform better than the Black-Scholes-Merton delta hedging strategy in markets with transaction costs?
- RQ3** How does the choice of agent architecture affect the stability and reliability of the hedging performance in non-perfect markets?

⁵See <https://www.investopedia.com/terms/b/blackscholes.asp>

2. Theory

This section explains the fundamental theory needed to understand the method explained in Section 3. Section 2.1 gives a financial background, where options, hedging, and the Black-Scholes-Merton model are introduced. Section 2.2 explains the core components of Reinforcement Learning (RL), including the agent, environment, reward, state, action, and policy. Section 2.3 explains the Markov decision process (MDP) and the Markov property. Section 2.4 explains the state-value and action-value functions, as well as the reward function. Section 2.5 goes into how the state and action value functions are connected, as well as how they can be explained recursively with respect to each other. This is also known as the Bellman equation, which is one of the core concepts in reinforcement learning. Finally, Section 2.6 introduces techniques to estimate the Bellman optimality.

2.1. Financial Background

Reinforcement learning can be applied in any field where decision-making and control over time are essential. In this thesis, it is applied to financial risk management, concerning option hedging. While this financial background is not essential for understanding the reinforcement learning framework, the next sections provide additional context for interested readers. Section 2.1.1 will introduce the financial derivative options, Section 2.1.2 explain the risk-management tool hedging. Finally, Section 2.1.3 explains the widely used method to determine the fair value of an options contract, called the Black-Scholes-Merton model.

2.1.1. Options

Derivatives are financial contracts between two or more parties whose value is determined by the price movements of one or more underlying assets, such as stocks, bonds, or commodities. In financial markets, these instruments serve as vital mechanisms for leveraging positions, speculating on future price directions, or hedging against systemic and asset-specific risks⁶.

An option is a financial derivative that grants the right, but not the obligation, to either buy or sell an asset at a predetermined price⁷. Options may therefore be either call or a put option, and they are further classified according to the flexibility of their exercise date. One of the simplest variants is the European option, which only can be exercised at maturity. BODIE et al. (2011, p. 51–52) explains that a call option generates profit if the price of the underlying asset rises above the break-even point (strike price + premium paid) while a put option becomes profitable if the price falls below the break-even-point.

An investor can take one of two sides in an options contract. The buyer pays a premium upfront in exchange for the rights described above. The seller, also referred to as the option writer, receives this premium but assumes the obligation to fulfill the contract if the buyer

⁶See <https://www.investopedia.com/terms/d/derivative.asp>

⁷Also known as the strike price

chooses to exercise.

In this thesis, we have focused on a short European call position. This is the perspective of the option writer who has sold a call option. Unlike the buyer, whose maximum loss is limited to the premium paid, the option writer is exposed to theoretically unlimited losses if the underlying asset price rises above the strike price. This risk profile is illustrated in Figure 2⁸.



Figure 2: Pay-off diagram for a short call option.

The risk associated with holding this sort of financial derivative can be reduced through hedging, which will be explained in the following section.

2.1.2. Hedging

Hedging against investment risk refers to the practice of taking an offset position in the underlying asset to reduce exposure to adverse price movements⁹. For a short call position, the seller is exposed to losses when the asset price rises. By holding a long position in the underlying asset, these losses can be partially or fully offset. If the asset price rises, the gain from the long position compensates for the loss on the short call.

In practice, the relationship between the option value and the underlying asset price changes continuously as market conditions evolve. This means that the position must be dynamically readjusted over time. The proportion of the underlying asset held in the hedge at any given point in time is known as the hedge ratio¹⁰, and determining this ratio optimally is the central problem addressed in this thesis.

⁸See <https://www.adigitalblogger.com/option-strategy/short-call/>. Last accessed 12.05.26.

⁹See <https://www.investopedia.com/trading/hedging-beginners-guide/>. Last accessed 25.05.26

¹⁰See <https://www.investopedia.com/terms/h/hedgeratio.asp>. Last accessed 25.05.26

2.1.3. The Black-Scholes-Merton Model

A central challenge in this context is the accurate pricing and hedging of financial derivatives, such as options. A widely used method for pricing European options is the Black-Scholes-Merton (BSM) model. The BSM model is a mathematical equation that estimates the theoretical value of a derivative. The model provides an analytical framework for both pricing European options and constructing hedging strategies. The value of a call option under the BSM model is given by BODIE et al. (2011, p. 725, Equation 21.1):

$$C_0 = S_0 N(d_1) - X e^{-rT} N(d_2), \quad (1)$$

where

$$d_1 = \frac{\ln(S_0/X) + \left(r + \frac{\sigma^2}{2}\right) T}{\sigma \sqrt{T}}, \quad d_2 = d_1 - \sigma \sqrt{T}.$$

Here C_0 is the current call option value, S_0 is the current stock price, $N(d)$ is the cumulative distribution function of the standard normal distribution, X is the exercise price (strike), r is the risk free interest rate, T is the time to maturity and σ is the standard deviation of the continuously compounded return.

In addition to pricing, the BSM model can be used to calculate the hedge ratio, commonly referred to as the option's delta. Delta measures the sensitivity of the option price to changes in the underlying asset price.

Delta hedging involves holding a position in the underlying asset equal to the option's delta at each point in time, so that small changes in the asset price are approximately offset by corresponding changes in the hedge portfolio. BODIE et al. (2011, p. 734) states that the $N(d_1)$ in Equation (1) corresponds the hedge ratio for a call option. If the delta of a call option for example would be 0.5, the hedging strategy would consist of holding 0.5 units of the underlying asset per option contract. In this way, small changes in the asset price are approximately canceled out, reducing the overall risk of the position.

In BODIE et al. (2011, p. 725), it is presented that under the assumptions of the BSM model, which are continuous trading, constant volatility, and the absence of transaction costs, dynamic readjustment will result in a theoretically perfect hedge. However, these assumptions do not reflect real market conditions. Constant readjustment in the presence of transaction costs can make hedging expensive. To address these limitations, this thesis explores an alternative approach based on deep reinforcement learning, which has the potential to learn adaptive hedging strategies directly from interactions with the market environment.

2.2. Reinforcement Learning Framework

This section explains the fundamentals of Reinforcement Learning (RL), where Section 2.2.1 explain the main components of reinforcement learning, and Section 2.2.2 explain the policy, and how it is applied in RL.

2.2.1. Elements of Reinforcement Learning

Reinforcement Learning is a framework for modeling sequential decision-making problems. The binomial decision tree in Figure 1 is a clear and simple example of a sequential decision-making problem, but games like chess, self driving cars and robotics are all part of the same type of problems. Sequential decision-making is a process of making a series of interrelated choices over time, where each decision depends on previous actions and influences future options.

The fundamentals of reinforcement learning involve an *agent* that interacts with an *environment* by making *observations*, taking *actions*, and receiving *rewards*. This process is illustrated in Figure 3, and each component and step in the figure will be explained in detail later in this thesis.

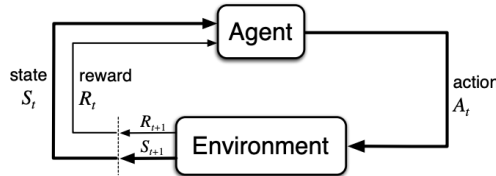


Figure 3: The fundamental methodology of RL, visualizing the state, agent, action, environment, and reward, taken from SUTTON and BARTO (2018, p. 48)

Figure 3 illustrates how the discrete timestep, t , is updated. The agent observes the current state, S_t , selects an action, A_t , and receives feedback in the form of a reward, R_t . As a consequence of the action, A_t , the environment transitions to a new state, S_{t+1} . This interaction continues over a sequence of time steps, and the end of a time sequence form an episode. Looking at the binomial decision tree in Figure 1, the agent would be the one deciding to move either up or down, and the state would be the current placement at time t in the tree. The environment would be the full dynamic of the tree, the action would be the next move (up or down), and the reward can in a simple example be given if the agent reach the winning state. The episode ends once a terminal state is reached. This interaction continues over multiple episodes and forms the basis of the learning process.

The environment related to option hedging will be explained in more detail in Section 3.1, and the agents are explained in Section 3.2. The goal of the agent is to learn a strategy, also known as a policy, that maximizes the cumulative reward over time. The way the reward is defined and evaluated will be described in Section 2.4, and how the policy is described in Section 2.2.2

2.2.2. Policy

The policy, denoted π , is the central objective in reinforcement learning, since it determines how the agent behaves in a given state. In practice, the policy is mapping states to actions and it can be seen as the strategy that the agent will follow. In KOLM and RITTER (2019, p. 161), they state that the agent ultimately searches for a policy that maximizes the expected value of all future rewards. The policy is formally defined by LAPAN (2018, p. 22) as the conditional probability distribution over actions for every possible state.

A stochastic policy provides a conditional probability distribution over the available actions for every possible state. It is formally defined by SUTTON and BARTO (2018, p. 58) as:

$$\pi(a|s) := \mathbb{P}[A_t = a|S_t = s], \quad (2)$$

where Equation (2) describes the probability that the agent chooses the action, a , given that it stands in state s .

Conversely, a deterministic policy maps a state directly to a single specific action. Because there is no probability distribution involved, the notation simplifies to:

$$\pi(s_t) = a_t,$$

which means the agent will always execute action a_t when encountering state s_t .

During reinforcement learning training, the policy is continuously updated as the agent interacts with the environment. In CAO et al. (2020, p. 5) they explain that to be able to learn an optimal policy, the agent must have a balance between exploration (discovering new states and actions) and exploitation (using known actions that yield high rewards). This is handled different depending on the agents architecture, and the agent specific exploration and exploitation are described together with the agent, later in the thesis.

2.3. Markov Decision Process

To analyze reinforcement learning problems, a mathematical framework is required. This is provided by the Markov Decision Process (MDP), which models sequential decision-making in environments with uncertain outcomes. In such environments, outcomes are partly controlled by a decision maker and partly determined by randomness. In reinforcement learning, MDPs form the theoretical foundation for representing the environment and defining the agent's objective ¹¹.

Within this framework, the agent acts as the decision maker and aims to learn a policy that maximizes the expected cumulative reward over time. A challenge related to this, noted

¹¹See <https://www.sciencedirect.com/topics/computer-science/markov-decision-process>. Last accessed 12.03.26

by SUTTON and BARTO (2018, p. 47), is that each action influences not only the immediate reward but also the state the agent will subsequently find itself in, and therefore all future rewards. Decision-making in MDPs involves a trade-off between immediate and delayed rewards.

2.3.1. The MDP Tuple (S, A, P, R)

In reinforcement learning, the agent interacts with an environment over a sequence of discrete time steps. At each time step t , the agent observes a state S_t , selects an action A_t , receives a reward R_{t+1} , and transitions to a new state S_{t+1} . This interaction generates a sequence known as a trajectory. Formally, an MDP is defined as a tuple:

$$(S, A, P, R)$$

where S is the set of all possible states, A denotes the set of all possible actions, P denotes the transition probabilities of moving from one state to another given a specific action, and R represents the numerical reward the agent receives after taking an action. The sets define the boundaries of the world that the agent lives in.

The dynamics of a MDP is described by SUTTON and BARTO (2018, p. 48, Equation 3.2) as the probability of transitioning to a new state, s' and receiving a reward, r given the current state, s and action, a :

$$p(s', r \mid s, a) = \mathbb{P}(S_t = s', R_t = r \mid S_{t-1} = s, A_{t-1} = a). \quad (3)$$

In a finite MDP, the sets of states, actions, and rewards, denoted by $\mathcal{S}$, $\mathcal{A}$, and $\mathcal{R}$, respectively, contain a finite number of elements. The variables R_t and S_t are treated as random variables, and $s' \in \mathcal{S}$ denotes the next possible state.

2.3.2. Markov Property

A key assumption underlying MDPs is the Markov property. As described by SUTTON and BARTO (2018, p. 49), this means that the future evolution of the system depends only on the current state and action, and not on any past history. Formally, this implies that the conditional probability of the next state S_{t+1} and reward, R_{t+1} , depends only on the present state, S_t , and action, A_t , as seen in Equation (3). This assumption significantly simplifies the learning problem, as the agent does not need to consider the entire sequence of past events when making decisions.

2.4. Return and Value Functions

If we return to the example of the binomial tree in the introduction, we could evolve the example and add rewards at each time t , this is visualized in Figure 4. We want to maximize the reward, and the best short-term action would be to take the action down in the first state. However, the objective is to find a policy that maximizes the long term reward, and the optimal policy would in this example be to move up twice.

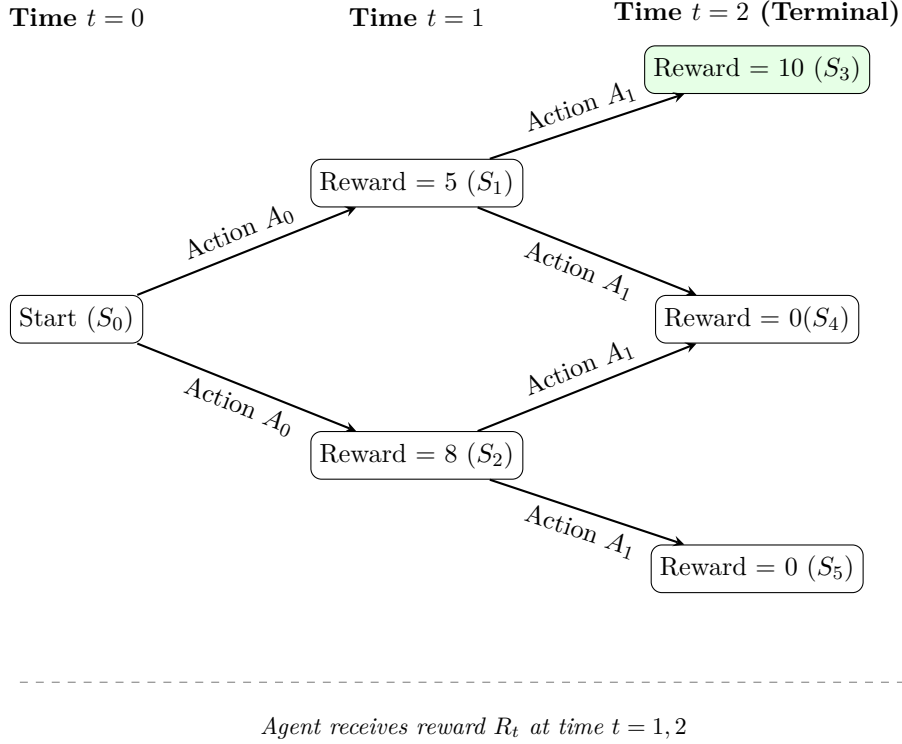


Figure 4: A simple two-step decision tree representing an agent navigating an environment through sequential choices and receiving rewards at each time t .

The objective of the agent is to find a policy that maximizes the cumulative reward over time. This is done by choosing actions that yield the most return at the end of an episode, and the natural question is: how do we evaluate how good a particular state or action is? This is done by the reward signal and the value function. These functions quantify return and form the foundation for decision-making in reinforcement learning.

2.4.1. Return and Reward Signal

To be able to determine how well a policy operates, the agent evaluates the reward signal. In SUTTON and BARTO (2018, p. 6), the reward signal is specified as the objective of the learning process by providing feedback on the agent's actions. The reward is received after the action is taken, and it's given as a numerical value. The reward signal indicates which outcomes are desirable or undesirable and guides the learning process, where a high reward is desirable, while a low reward is undesirable. If an action results in a low or unfavorable reward, the policy may be updated so that the agent selects a different action in similar

situations in the future.

The reward function is, in contrast, explained by SUTTON and BARTO (2018, p. 6) as a function that evaluates the long-term value of states or actions by estimating the expected cumulative future reward. The success of an RL agent depends on designing a reward function that accurately reflects the desired behavior. The reward signal, defined as R_{t+1} , provides immediate feedback to the agent after taking an action. However, the overall objective is not to maximize individual rewards, but the cumulative reward over time, which is referred to as the discounted sum of future rewards, G_t .

The return at time step t is defined by SUTTON and BARTO (2018, p. 55, Equation 3.8) as:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{T-1} R_T, \quad (4)$$

where T is the terminal time and $\gamma \in (0, 1]$ is the discount factor. The discount factor plays an essential role in the reward function (4) as it determines how much importance the agent assigns to future rewards compared to immediate rewards. SAMI (2020, p. 9) states that a γ close to one places greater emphasis on long-term rewards, encouraging the agent to consider the future consequences of its actions rather than focusing only on short-term gain. Consequently, a γ close to zero causes the agent to focus more strongly on immediate rewards, which results in a more short-sighted policy.

If we look at the example in Figure 4 and set $\gamma = 0.1$, the maximum reward would be given by taking the action to move down, and collect the reward of 8, compared to moving up and collecting a reward of only 6. If we on the other hand set $\gamma = 0.9$, the cumulative reward at time would be 14 by moving up for both actions compared to 8 by moving down.

2.4.2. Value Functions

The framework for a Markov Decision Process is defined by the state-value function, $v_\pi(s)$, as the expected value of the future cumulative reward, when an agent is in state s and follows the policy π . The state-value function is defined by SUTTON and BARTO (2018, p. 70, Equation 3.12) as:

$$v_\pi(s) = \mathbb{E}_\pi[G_t | S_t = s], \quad (5)$$

where G_t is the reward function (4) and $\mathbb{E}_\pi[.]$ denotes the expected return of a random variable given that the agent follows policy π , at any time step t .

While the state-value function evaluates states, the action-value function evaluates the state-action pairs. The action-value function is also referred to as the Q-value. It is defined as the expected value of the future cumulative reward, if the agent takes an action, a , from state, s , and follows the policy π . It is defined by SUTTON and BARTO (2018, p. 70, Equation 3.13)

as:

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a], \quad (6)$$

where G_t is the reward function (4). $\mathbb{E}_\pi[\cdot]$ denotes the expected return of a random variable given that the agent follows policy π , at any time step t .

Connection between the Value functions

The state- and action- value functions are closely related. The value of a state can be expressed as the expected value of the action-value function under the given policy:

$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) q_\pi(s, a), \quad (7)$$

where $\mathcal{A}$ is the set of all possible actions available in state s . Hence, the value function can be expressed as the return conditional of the current space, action, weighted by the probability of taking each action due to the law of total probability.

2.5. The Bellman Equation

Although value function provides a way to evaluate states and action, computing them directly is often infeasible, especially in complex environments. Instead, value function can be expressed recursive in complex environments using the Bellman equation, which forms the basis of all the reinforcement learning algorithms that are implemented in this thesis. It allows us to convert information of the environment into improvements of the agents behavior. The Bellman equation breaks down the state in the initial reward plus the discounted value of the later rewards.

To derive the Bellman equation for the state-value function, we begin with the definition of the reward function (4):

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots,$$

this can be rewritten as

$$G_t = R_{t+1} + \gamma G_{t+1}. \quad (8)$$

By definition, the state-value function $v_\pi(s)$ (5) is the expected return starting from state s under policy π :

$$v_\pi(s) = \mathbb{E}_\pi[G_t | S_t = s] \quad (9)$$

Substituting the recursive definition of the return from Equation (8) into Equation (9) we obtain:

$$v_\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} | S_t = s]$$

To explicitly account for the choices made by the agent, we apply the law of total expectation to condition the expression on the immediate action $A_t = a$ taken according to the policy in

Equation (7):

$$v_\pi(s) = \sum_a \pi(a | s) \underbrace{\mathbb{E}_\pi[G_t | S_t = s, A_t = a]}_{q_\pi(s,a)}. \quad (10)$$

In the q-function (10), we substitute the recursive expression for the return from Equation (8):

$$\mathbb{E}_\pi[G_t | S_t = s, A_t = a] = \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} | S_t = s, A_t = a].$$

The state-value function then looks like this:

$$v_\pi(s) = \sum_a \pi(a | s) \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} | S_t = s, A_t = a]. \quad (11)$$

Using the linearity of expectation, we can distribute the conditional expectation across the immediate reward and the future return terms within Equation (11):

$$v_\pi(s) = \sum_a \pi(a | s) (\mathbb{E}_\pi[R_{t+1} | S_t = s, A_t = a] + \gamma \mathbb{E}_\pi[G_{t+1} | S_t = s, A_t = a]).$$

The first term corresponds to the expected immediate reward, while the second term represents the expected future return. Using the Markov property, the future return G_{t+1} depends only on the state S_{t+1} . Therefore, $\mathbb{E}_\pi[G_{t+1}]$ is equivalent to the state-value function of the successor state, $v_\pi(S_{t+1})$. This allows us to simplify the expression to:

$$v_\pi(s) = \sum_a \pi(a | s) \mathbb{E}_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s, A_t = a].$$

Finally, we expand the remaining expectation over the environmental dynamics. The joint probability of transitioning to a next state s' and receiving an immediate reward r given the current state s and action a is defined by the four-argument conditional probability distribution $p(s', r | s, a)$, from Equation (3). Replacing the abstract expectation with a summation over these transition dynamics yields the standard Bellman expectation equation for the state-value function:

$$v_\pi(s) = \sum_a \pi(a | s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_\pi(s')]. \quad (12)$$

This illustrates that the value of a state is fundamentally tied to the immediate rewards of available actions and the values of neighboring states. This recursive structure forms the mathematical backbone for value-based reinforcement learning algorithms.

2.5.1. Bellman Optimality

The primary objective in reinforcement learning is to discover a policy that maximizes the expected cumulative reward over time. For any finite Markov Decision Process, SUTTON and BARTO (2018, p. 62) states that there always exists at least one optimal policy, denoted by π_* , which achieves the highest possible value across all states.

The optimal state-value function, $v_*(s)$, represents the maximum expected return achievable from state s . It is defined by SUTTON and BARTO (2018, p. 63) as:

$$v_*(s) = \max_{\pi} v_{\pi}(s), \quad (13)$$

for all $s \in \mathcal{S}$. Similarly, the optimal action-value function, $q_*(s, a)$, represents the expected return of taking action a in state s and thereafter following an optimal policy. It is defined as:

$$q_*(s, a) = \max_{\pi} q_{\pi}(s, a),$$

for all $s \in \mathcal{S}, a \in \mathcal{A}$. Because v_* and q_* represent optimal behavior, they satisfy the self-consistency conditions known as the Bellman optimality equations. For the state-value function, we insert the optimal state value function (13) into the standard expectation equation for the Bellman Equation (12). The Bellman optimality equation is given by SUTTON and BARTO (2018, p. 64, Equation. 3.19):

$$v_*(s) = \max_{a \in \mathcal{A}} \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_*(s')]. \quad (14)$$

This equation establishes that the value of a state under an optimal policy must equal the expected return from the best immediate action, assuming optimal behavior in all subsequent states.

The corresponding Bellman optimality equation for the action-value function is given by SUTTON and BARTO (2018, p. 64, Equation. 3.20):

$$q_*(s, a) = \sum_{s', r} p(s', r \mid s, a) \left[r + \gamma \max_{a'} q_*(s', a') \right]. \quad (15)$$

This formulation recursively expresses the value of an action-state pair by taking the expected sum of the immediate reward and the discounted maximum value achievable from the resulting next state s' .

These equations are fundamental to RL because they provide a recursive characterization of optimality without requiring explicit knowledge of the optimal policy itself. In principle, solving either equation yields an optimal policy for a given MDP.

However, obtaining an exact solution analytically or computationally is rarely feasible in practice. In SUTTON and BARTO (2018, p. 66), it is stated that an exact solution relies on three assumptions:

1. The transition dynamics of the environment, $p(s', r \mid s, a)$, must be perfectly known.
2. The environment must satisfy the Markov property.
3. Enough computational power.

While the first two conditions can be satisfied in well-defined problems, the bottleneck typically shifts to the third requirement. SUTTON and BARTO (2018, p. 66) note that computing the Bellman optimality equation for the board game Backgammon would require thousands of years of computation on the fastest supercomputers available in 2018 just to resolve the state-value function and thousands of years for the action-value function. In large or continuous state spaces exact computation is impossible. Reinforcement learning overcomes this by approximating value functions from sampled experience, rather than computing exact expectations over all possible states and transitions.

2.6. Estimation of the Bellman Equations

Several approaches have been developed to approximate the value functions implied by the Bellman equations. In the following sections, Dynamic Programming, Monte Carlo methods, Temporal Difference learning, and Q-learning are introduced. These methods represent different techniques for estimating Bellman and form the basis for many reinforcement learning algorithms.

2.6.1. Dynamic Programming

Dynamic Programming (DP) refers to a set of methods used to find optimal policies which have complete knowledge of the environment, as an MDP. In other words, DP is used to solve reinforcement learning problems when a complete model of the environment is available. In particular, DP assumes that the transition probabilities $p(s', r \mid s, a)$ and the reward function (4) are known. Under these assumptions, the Bellman equations can be solved exactly.

Two central components of dynamic programming are **policy evaluation** and **policy improvement**. The two components are described by SUTTON and BARTO (2018, p. 74–80) like the following; policy evaluation consists of computing the state-value function $v_\pi(s)$ for a given policy π , typically by iteratively applying the Bellman expectation equation until convergence. Policy improvement, on the other hand, updates the policy by selecting actions that maximize the expected return based on the current value function.

By alternating between policy evaluation and policy improvement, the policy can be iteratively refined. This process is described by SUTTON and BARTO (2018, p. 96) as **policy iteration** and is guaranteed to converge to an optimal policy in finite Markov Decision Processes.

$$\pi_0 \xrightarrow{\mathbb{E}} v_{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{\mathbb{E}} v_{\pi_1} \xrightarrow{I} \pi_2 \xrightarrow{\mathbb{E}} \dots \xrightarrow{I} \pi_* \xrightarrow{\mathbb{E}} v_{\pi_*},$$

where $\xrightarrow{\mathbb{E}}$ denotes the policy evaluation and $\xrightarrow{I}$ denotes the policy improvement.

An alternative approach in DP is **value iteration**, which directly applies the Bellman optimality equation to update the value function. Instead of evaluating a fixed policy, value iteration combines evaluation and improvement into a single update step. This is defined by SUTTON and BARTO (2018, p. 100, Equation 4.10) as:

$$v_{k+1}(s) = \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_k(s')].$$

While Dynamic Programming provides a theoretically sound framework for solving reinforcement learning problems, it is rarely applicable in practice. The requirement of a known environment model and the need to perform updates over the entire state space make DP infeasible for large or complex problems.

These limitations motivate the development of model-free methods, which can learn directly from experience without requiring knowledge of the environment dynamics. Such approaches are introduced in the following sections.

2.6.2. Monte Carlo Methods

Monte Carlo (MC) methods address some of the key limitations of dynamic programming by removing the requirement of a known environment model. In many real-world problems, the model would need to perform updates on the entire state space, making it unachievable in large, and complex problems. SUTTON and BARTO (2018, p. 91) state that instead of relying on transition probabilities, MC methods learn value functions directly from sampled experience.

In MC methods, value estimates are based on complete episodes of interaction with the environment. After an episode has terminated, the return G_t can be computed for each visited state, and the value function is updated as an average of the observed returns. In this way, MC methods learn directly from trajectories generated by following a policy. This approach makes MC methods model-free, as they do not require knowledge of the transition dynamics $p(s', r \mid s, a)$. Instead, they rely solely on observed trajectories generated by following a policy.

A common approach is the every-visit Monte Carlo method, in SUTTON and BARTO (2018, p. 144, Equation 6.1) the value function is updated as:

$$V(S_t) \leftarrow V(S_t) + \alpha [G_t - V(S_t)],$$

where α is a constant step-size parameter.

CAO et al. (2020, p. 7) state that the key characteristic of Monte Carlo methods is that updates can only be performed after an episode has ended, since the return requires knowledge of all future rewards within that episode. This is visualized in Figure 5. This can lead to high variance in the estimates, especially in problems with long episodes, as the return depends on the entire sequence of rewards. Despite this, MC methods have the advantage of producing unbiased estimates of the value function, since they use actual observed returns rather than bootstrapped estimates.



Figure 5: One Monte Carlo episode, visualizing how returns are assigned to visited states after an episode has terminated.

In summary, Monte Carlo methods provide a conceptually simple approach to learning value functions from experience. However, their reliance on complete episodes and the resulting high variance motivate the development of more efficient methods. This leads to Temporal Difference learning, which combines ideas from both Dynamic Programming and Monte Carlo methods.

2.6.3. Temporal Difference Learning

Temporal Difference (TD) learning combines key ideas from both dynamic programming and Monte Carlo methods. Like Monte Carlo methods, TD learning is model-free and learns directly from experience. SUTTON and BARTO (2018, p. 74) states that it at the same time, incorporates the recursive structure of the Bellman equation, similar to dynamic programming.



Figure 6: One-step TD iteration

In SUTTON and BARTO (2018, p. 119) it is noted that TD learning does not need to wait until the end of an episode to update its estimates like Monte Carlo. Instead, TD methods can learn after a defined time step. TD is similar to dynamic programming, where value functions are updated using other value estimates derived from the Bellman equation. The main idea of TD learning is to update value estimates based on other learned estimates, rather than waiting for the final outcome of an episode like Monte Carlo, visualized in Figure 6. This is known as *bootstrapping*.

Instead of using the full return G_t , TD methods use a one-step estimate of the return:

$$R_{t+1} + \gamma v_\pi(S_{t+1}).$$

This leads to TD(0), or *one-step* TD, which is the simplest TD-method. The update rule for the state-value function is defined by SUTTON and BARTO (2018, p. 144, Equation 6.1) as:

$$v_\pi(S_t) \leftarrow v_\pi(S_t) + \alpha \left[\underbrace{R_{t+1} + \gamma v_\pi(S_{t+1})}_{\text{estimate of the return}} - v_\pi(S_t) \right],$$

where $\alpha \in (0, 1]$ is the learning rate.

The term in brackets is defined by SUTTON and BARTO (2018, p. 178, Equation 7.10),

$$\delta_t = R_{t+1} + \gamma v_\pi(S_{t+1}) - v_\pi(S_t),$$

is referred to as the *temporal difference error*. It represents the difference between the predicted value and a one-step improved estimate.

Compared to Monte Carlo methods, TD learning has lower variance, since it does not rely on full episode returns. However, this comes at the cost of introducing some bias, as the updates depend on current value estimates. In summary, TD learning provides a more efficient and practical approach to estimating value functions, particularly in problems with long or continuing tasks. This framework forms the basis for many modern reinforcement learning algorithms, including Q-learning, which is introduced in the next section.

2.6.4. Q-learning

Q-learning is explained by WATKINS (1989, p. 81) as a model-free reinforcement learning algorithm used to learn the optimal action-value function, commonly referred to as the Q-value. Unlike the methods introduced in the previous sections, which evaluate a fixed policy, Q-learning aims to approximate the optimal policy by iteratively improving its estimates of the Q-values using the Bellman optimality equation:

$$q_*(s, a) = \mathbb{E} \left[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') \mid S_t = s, A_t = a \right].$$

In practice, Q-learning approximates this equation using sampled experience rather than requiring full knowledge of the environment dynamics. At each time step, the Q-value for the current state-action pair is updated according to SUTTON and BARTO (2018, p. 131, Equation 6.8) as :

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t) \right],$$

where $\alpha \in (0, 1]$ is the learning rate.

The term

$$R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a')$$

is referred to as the target, and represents a one-step approximation of the optimal return. The difference between this target and the current estimate defines the temporal difference error explained in the previous section. Through repeated interaction with the environment, the Q-values are gradually adjusted toward the target, allowing the agent to improve its policy over time.

As noted by WATKINS and DAYAN (1992, p. 281), Q-learning stores these state-action values in a *Q-table*, which is continuously updated as the agent interacts with the environment. Although this approach is effective for small and discrete state-action spaces, it becomes impractical as the dimensionality of the problem increases or when the state space is continuous. This limitation motivates the introduction of deep reinforcement learning methods, where neural networks are used to approximate the Q-function instead of storing it explicitly in a table. The four agents implemented in this thesis are all built upon this principle, and their architectures are presented in detail in Section 3.2.

3. Method

This section presents the methodology used in this study. It describes how the environment and the agent are implemented, as well as how the training and evaluation procedures are conducted. The overall objective is to learn an optimal hedging strategy for a short position in an European call option.

3.1. Environment

The first step in the implementation is to define the environment in which the agent operates. After this we explain how the environment evolves from the current state to the next.

3.1.1. Environment Definition

As illustrated in Figure 3, the input to the environment is an action A_t , while the output is a state S_{t+1} and a reward R_{t+1} . These components are defined as follows:

- **Action:** The action represents the hedge ratio and corresponds to a value between zero and one, indicating the proportion of the underlying asset held. Zero means that the agent holds nothing of the underlying asset and is completely unhedged. One means the agent holds a full position, completely offsetting the option exposure. The decision of how much to hedge is made once per day over the lifetime of the option, which in this thesis is three months.
- **State:** The state consists of three variables observed by the agent: Moneyness¹², time to maturity (TTM), and the current position in the underlying asset.
- **Reward:** The reward is defined using the same accounting profit and loss (P&L) formulation as CAO et al. (2020, p. 11–12). At each time step, it is given by the change in portfolio value, adjusted for gains or losses from the hedging position and transaction costs:

$$R_{i+1} = V_{i+1} - V_i + H_i(S_{i+1} - S_i) - \kappa|S_i(H_{i+1} - H_i)|, \quad (16)$$

for $0 \leq i < n$, where S_i denotes the asset price at the beginning of period i , and H_i represents the position held in the underlying asset over the interval $[i\Delta t, (i+1)\Delta t]$. The parameter κ captures proportional transaction costs, while V_i is the value of the derivative position at time i and $|\cdot|$ denotes the absolute value. In addition to the stepwise rewards, transaction costs are also incurred when initiating and closing the hedge. The initial cost of establishing the position is given by $-\kappa|S_0H_0|$, and a corresponding cost is applied when the position is liquidated at the final time step: $-\kappa|S_nH_n|$.

¹²Moneyness is the relationship between an option's strike price and the current market price of its underlying asset, indicating whether an option would be profitable if exercised immediately. It determines if an option is in-the-money (ITM), at-the-money (ATM), or out-of-the-money (OTM). See <https://www.investopedia.com/terms/m/moneyness.asp>. Last accessed 2026-04-09.

3.1.2. Environment Evolution

Now that all variables have been defined, the next step is to describe how the environment evolves at each time step. Since we assume that the agent readjust its hedging positions in discrete time, the Geometric Brownian Motion (GBM) model by HUANG and LAWRYSHYN (2025, p. 12, Equation. 14) as following:

$$S(t) = S(0) \exp\left(\left(\mu - \frac{\sigma^2}{2}\right)t + \sigma Z\sqrt{t}\right).$$

Here μ is the expected annual return of the stock and σ is the annual volatility. $Z_t \sim N(0, 1)$ is a standard random normal variable and t is a discrete time step, specifically 1 day. The initial assumptions we have are: constant volatility, $\sigma = 0.2$, the initial stock price, $S_0 = 100$ and $\mu = 0.05$ with a time to expiry of 3 month (63 trading days). The reason for these initials is to be able to compare towards Matworks "Hedge Options Using Reinforcement Learning ToolBox" ¹³. The key reasons for using the GBM in our model evaluation are that it is non-negative, which is necessary because the prices of the underlying asset cannot be negative, and it assumes the Markov property.

The reward is computed based on the accounting profit and loss formulation in Equation (16). The change in the value of the derivative is calculated using the Black-Scholes formula. To account for risk preferences, a quadratic penalty term is applied to the profit and loss, resulting in the final reward: $R_t = \text{P\&L} - c \cdot (\text{P\&L})^2$, where c is a risk-aversion parameter that penalizes large fluctuations in returns¹⁴.

Finally, the environment updates its internal state by setting the new asset price, time to maturity, and position equal to the chosen action. The updated state, the computed reward, and a termination signal are then returned to the agent.

3.2. Agents

After the environment is defined, we turn to the decision-making component of the system: the agent. In the context of option hedging, the agents objective is to make sequential decisions about the hedge position in the underlying asset in order to minimize cost. To address the complexities of non-perfect markets, four agents are implemented using deep reinforcement learning: the Deep Q-Network (DQN), the Deep Deterministic Policy Gradient (DDPG), the Twin Delayed Deep Deterministic Policy Gradient (TD3), and a Hybrid agent that combines DQN and TD3.

In Section 2, the fundamentals to reinforcement learning were introduced, but as we have extended our work to Deep Reinforcement Learning (DRL), which is explained by FRANÇOIS-

¹³See <https://se.mathworks.com/help/finance/hedging-option-using-reinforcement-learning.html>. Last accessed 09.06.26

¹⁴See <https://se.mathworks.com/help/finance/hedging-option-using-reinforcement-learning.html>. Last accessed 2026-01-22.

LAVET et al. (2018) as a combination of reinforcement learning (RL) and deep learning, some new concepts will be introduced.

The agents are presented in order of increasing complexity. Each agent builds on its predecessor while introducing modifications to address specific limitations. The DQN is the natural starting point, as it extends the Q-learning framework introduced in Section 2.6.4 directly to deep neural networks. The DDPG then generalises this to a continuous action space. The TD3 addresses known instabilities in DDPG, and the Hybrid agent combines the DQN and TD3 within a multi-agent framework.

3.2.1. Deep Q-Network (DQN)

The Deep Q-Network (DQN) is the most direct extension of the Q-learning algorithm described in Section 2.6.4. In Q-learning, a table of Q-values is maintained for every state–action pair and updated iteratively using sampled experience. While this approach works well for problems with small and discrete state spaces, it becomes impractical as the state space grows larger or becomes continuous.

In this thesis, the state consists of moneyness, time to maturity, and the current hedge position, forming a continuous three-dimensional space in which a table would need infinitely many entries. The solution is to replace the Q-table with a neural network that approximates the Q-function across all states. The structure and purpose of this network are described below.

Q-network

As described by GÉRON (2017, p. 633), the DQN agent uses a neural network, referred to as the Q-network, to approximate the optimal action-value function: $Q(s, a; \theta)$, where θ denotes a set of parameters. The network takes the current state as input and outputs a Q-value for each action in a discrete action set. In this thesis the action set is in the range $\in [0, 0.1, \dots, 0.9, 1]$ where each value represents the proportion of the underlying asset held in the hedge. The network therefore output a vector of eleven values and the agent selects the action with the highest predicted return.

The network is updated by minimizing the squared error between the estimated Q-Values $Q(s, a, \theta)$ and the a target Q-value, derived from the Bellman optimality equation. The loss function we want to minimize, which is found in the work of MESSAOUD et al. (2020, p. 33, Equation 29), is seen below:

$$\mathcal{L}(\theta, D) = \mathbb{E}_{(s, a, r, s', d) \sim \mathcal{D}} \left[\left(\underbrace{Q(s, a; \theta)}_{\text{Estimated value}} - \underbrace{(r + \gamma(1 - d) \max_{a'} Q(s', a'; \theta_{targ}))}_{\text{Target value}} \right)^2 \right]. \quad (17)$$

The target value consists of the immediate reward r together with the discounted estimate of the future rewards. Here, θ_{targ} denotes the parameters of a separate target network, which is a delayed copy of the main network and is used to improve stability. So instead of updating the target network directly after each iteration, its parameters are updated according to the equation below, as described by LILLICRAP et al. (2015, p. 4),

$$\theta_{targ} = \tau\theta + (1 - \tau)\theta_{targ}, \quad (18)$$

where $\tau \in [0, 1]$ is a hyperparameter. This procedure, commonly referred to as a soft update, slowly adjusts the target network toward the parameters of the main network, resulting in smoother and more stable learning.

The term d in the loss function indicates whether the next state is terminal ($d = 1$ if episode has ended, 0 otherwise), while $\mathcal{D}$ denotes a mini-batch sampled from the replay buffer. As introduced by MNIH et al. (2013, p. 4), the replay buffer stores experience tuples (s, a, r, s', d) at each time step as the agent interacts with the environment. During training, random batches of these past experiences are sampled and used to update the network parameters. MNIH et al. (2013, p. 4–5) further explain that the replay buffer improves data efficiency by allowing the agent to reuse past experiences multiple times. In addition, random sampling from the buffer breaks the correlations between the samples, which reduces the variance of the updates.

The loss function in Equation (17) is minimized using the Adam optimizer, a gradient based optimization method, as described by KINGMA and BA (2015, p. 1). Gradients of the loss with respect to the network parameters are computed via backpropagation and used to iteratively update the model weights.

Epsilon-greedy

The DQN algorithm is an off-policy reinforcement learning method. As described by SUTTON and BARTO (2018, p. 103), this means that the agent learns one policy, referred to as the target policy, while following a different policy, known as the behavior policy, to generate data. This separation offers greater flexibility when dealing with the trade-off between exploration and exploitation, making off-policy methods more powerful and general. However, SUTTON and BARTO (2018, p. 103) also note that this comes at the cost of greater variance and slower convergence compared to on-policy methods.

In DQN, exploration is handled using an epsilon-greedy policy, denoted as π_ϵ . As defined by CAO et al. (2020, p. 9), under this strategy, the agent follows a greedy policy with a probability of $1 - \epsilon$ and a random policy with probability ϵ , which gives us:

$$\pi_\epsilon(S_t) = \begin{cases} \arg \max_{a \in A} Q(S_t, a), & \text{with probability } 1 - \epsilon, \\ \text{random } a \in A, & \text{with probability } \epsilon. \end{cases}$$

In the beginning of the training our ϵ is set to 1, meaning the agent is very exploratory. As the training continues the epsilon is decreased, leading the agent to rely more on the learned policy. In the end of the learning procedure the greedy policy is always followed, and at last we get our optimal policy.

3.2.2. Deep Deterministic Policy Gradient (DDPG)

The deep deterministic policy gradient (DDPG) algorithm addresses the main limitation of DQN by extending deep Q-learning to continuous action spaces. Rather than selecting from a fixed discrete set, the DDPG agent can output any hedge ratio between $[0, 1]$.

This extension requires a structural change. In DQN, the Q-network outputs one value per action, so the greedy action can be found by simply taking the maximum over all outputs. In a continuous action space this is no longer possible. As described by SUMIEA et al. (2024, p. 2), the DDPG resolves this by introducing an actor-critic architecture, in which two neural networks divide the task:

- The critic, estimates the Q-value function and evaluates how good a chosen action is.
- The actor, represents the policy and determines which action to take given a state.

This structure enables DDPG to combine policy learning with value estimation. In the next two parts, the critic and then actor network are explained in more detail.

Critic Network

The critic network takes in the current state and action as input and then outputs the corresponding Q-value, following SUMIEA et al. (2024, p. 4). In this thesis the neural network for the critic was built using three fully connected layers and Rectified Linear Unit (ReLU) as activation functions.

The critic network is updated by minimizing the difference between the current estimated Q-value from the critic and a target constructed from both a target critic and a target actor. The loss function that we want to minimize, is defined by SUMIEA et al. (2024, p. 4, Equation 7):

$$L(\phi, \mathcal{D}) = \mathbb{E}_{(s,a,r,s',d) \sim \mathcal{D}} \left[\left(\underbrace{Q_\phi(s, a)}_{\text{Estimated value}} - \underbrace{(r + \gamma(1-d)Q_{\phi_{\text{targ}}}(s', \mu_{\theta_{\text{targ}}}(s')))}_{\text{Target value}} \right)^2 \right], \quad (19)$$

where d indicates whether the episode has terminated, and $\mathcal{D}$ is a mini-batch sampled from the replay buffer. Q_ϕ denotes the critic neural network, with parameters ϕ and $\mu_{\theta_{\text{targ}}}$ represents the target policy. The target networks one for the critic ϕ_{targ} and one for the actor θ_{targ} is updated according to Equation (18). The loss function from Equation (19) is minimized using the Adam optimizer.

Actor Network

The actor network takes in the current state and outputs an action, following SUMIEA et al. (2024, p. 4). The neural network for the actor was built using three fully connected layers. ReLU activation functions are used in the hidden layers, while a sigmoid activation function is applied in the output layer to ensure that the action remains within the desired range 0 to 1.

The actor is updated using policy gradients, which optimize the parameters of a policy by following the gradient in the direction of higher expected rewards. More specifically, it is trained to maximize the expected Q-value predicted by the critic, as defined in the work of SUMIEA et al. (2024, p. 4, Equation 8):

$$\max_{\theta} \mathbb{E}_{s \sim \mathcal{D}} [Q\phi(s, \mu_{\theta}(s))] .$$

This means that for each state in the mini-batch, the actor proposes an action, the critic evaluates it, and the actor's parameters are updated to increase the critic's predicted return. As defined in CAO et al. (2020, p. 11), the two networks are trained iteratively until convergence: the critic first, to provide feedback, and then the actor, to improve the policy in the direction the critic indicates.

Ornstein-Uhlenbeck Noise

The policy in DDPG is deterministic, meaning that a specific action is selected for each given state. While this allows for stable policy learning, it also limits exploration, as the agent will repeatedly choose the same action for the same state. To address this, noise is added to the action during training which encourage exploration. Several noise processes can be used for this purpose, and in this thesis, the Ornstein-Uhlenbeck (OU) process is applied.

The OU process is stochastic and in a discrete-time setting, LAPAN (2018, p. 412) defines the process as:

$$x_{t+1} = x_t + \theta(\mu - x_t) + \sigma N, \tag{20}$$

where θ determines the rate of mean reversion, μ is the long-term mean, σ controls the magnitude of the noise, and N is normal standard noise. As seen in Equation (20), the generated noise depends on its previous value x_t , which creates a smoother and more consistent exploration pattern compared to independent random noise.

All components of the DDPG algorithm have now been introduced. An overview of how these components interact during training is illustrated in Figure 7.

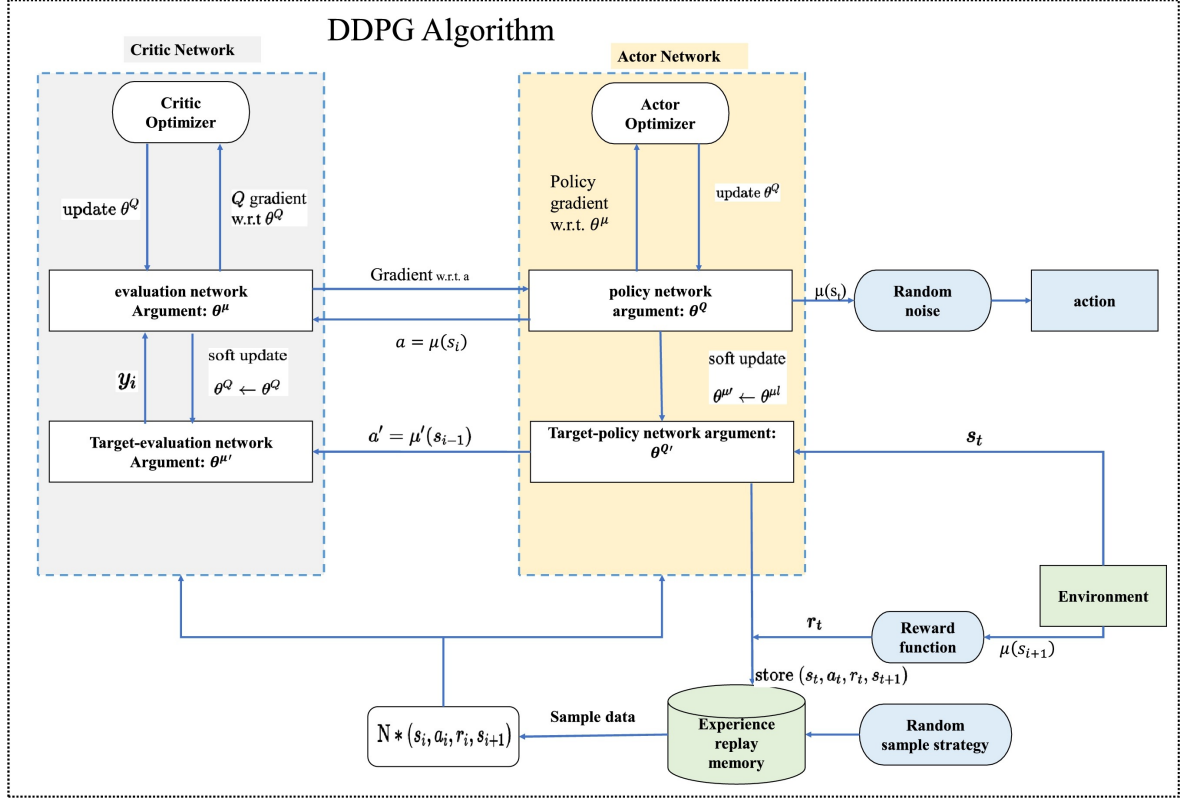


Figure 7: An overview of the structure for the DDPG algorithm, source: SUMIEA et al. (2024, p. 3).

While DDPG is a powerful tool capable of handling high-dimensional states and action spaces, SUMIEA et al. (2024, p. 2) mentioned limitations, including overestimation value estimates. This motivates the use of a modified version of DDPG, which is introduced in the following section.

3.2.3. Twin Delayed Deep Deterministic Policy Gradient (TD3)

The Twin Delayed Deep Deterministic Policy Gradient (TD3) algorithm builds upon the actor-critic framework used in DDPG but introduces several modifications to address known issues, such as overestimating Q-values and suboptimal policies. In particular, FUJIMOTO et al. (2018) propose three key improvements: clipped double Q-learning, target policy smoothing, and delayed policy updates.

Clipped double Q-learning

Clipped double Q-learning in TD3 involves learning two separate Q-functions, denoted Q_{ϕ_1} and Q_{ϕ_2} . Instead of relying on a single estimate, the minimum of the two Q-values is used when constructing the target in the loss function. Consequently, the target value from Equation (19) is modified according to FUJIMOTO et al. (2018, p. 4, Equation 10) work:

$$y(r, s', d) = r + \gamma(1 - d) \min_{i=1,2} Q_{\phi_i, \text{targ}}(s', a'(s')). \quad (21)$$

By using the smaller of the two estimates, the method produces more conservative targets, which helps against overestimation. Each Q-function is then trained independently by minimizing the mean squared error with respect to this shared target:

$$L(\phi_1, \mathcal{D}) = \mathbb{E} \left[(Q_{\phi_1}(s, a) - y(r, s', d))^2 \right],$$

$$L(\phi_2, \mathcal{D}) = \mathbb{E} \left[(Q_{\phi_2}(s, a) - y(r, s', d))^2 \right].$$

The policy is updated by maximizing the Q-value estimated by one of the critics, where we have used Q_{ϕ_1} :

$$\max_{\theta} \mathbb{E}_{s \sim \mathcal{D}} [Q_{\phi_1}(s, \mu_{\theta}(s))].$$

Target Policy Smoothing

In addition to clipped double Q-learning, TD3 applies target policy smoothing by adding small noise to the target action, denoted as $a'(s')$, when computing the target value in Equation (21). This noise is distinct from the exploration noise used in DDPG, which is added to the actor’s output during training, where we have used the OU process noise. In TD3, both types of noise are used, as they serve different purposes.

The target policy smoothing noise is introduced to prevent the policy from exploiting sharp errors in the critic and encourages smoother value estimates. After adding the noise, the resulting action is clipped to ensure it remains within the valid action range, $[0, 1]$. The target action is defined by FUJIMOTO et al. (2018, p. 6, Equation 14) as:

$$a'(s') = \mu_{\theta_{\text{targ}}}(s') + \epsilon, \quad \epsilon \sim \text{clip}(\mathcal{N}(0, \sigma), -c, c),$$

where $\mathcal{N}(0, \sigma)$ denotes a normal distribution with mean zero and standard deviation σ , and c controls the maximum magnitude of the smoothing noise.

Delayed Policy Updates

The third improvement proposed by FUJIMOTO et al. (2018, p. 5) in TD3 is the use of delayed policy updates. In the DDPG algorithm, the actor and critic networks are updated at the same frequency. In contrast, TD3 updates the actor network less frequently than the critic networks. This allows the value estimates to stabilize before being used to update the policy, reducing variance in the learning process.

3.2.4. Hybrid Agent

In addition to the three single-agent approaches, a Hybrid agent is explored in which two agents jointly determine the final action. As illustrated in Figure 8, the Hybrid agent combines a DQN and a TD3 agent in a hierarchical structure inspired by the work of HE et al. (2025), who applied a similar architecture to autonomous robot navigation.

In this architecture, the action selection process is divided into two steps. First, the DQN acts as a high-level decision maker, selecting a coarse bin from the discrete action set in range $[0, 0.1, \dots, 0.9, 1]$. This bin defines an interval $[a_{low}, a_{high}]$ of width 0.1. The TD3 agent then acts as a low-level controller, outputting a continuous value $u \in [0, 1]$, which is rescaled to the DQN-selected interval according to:

$$a = a_{low} + u(a_{high} - a_{low}).$$

The final hedge ratio a is therefore a continuous value, combining the structured exploration of a discrete policy with the precision of a continuous one. This hierarchy is intended to give the agent cost-awareness at two levels: the DQN determines when and to which broad region to move, while the TD3 refines this decision.

Two training variants of the Hybrid agent were explored: one in which the DQN and TD3 are trained simultaneously, and one in which they are trained sequentially (the DQN first, followed by TD3 fine-tuning within the selected bins).

A key challenge in this type of multi-agent setup is the credit assignment problem. As discussed by ALBRECHT et al. (2024, p. 106), when several agents act jointly, the resulting reward reflects the combined effect of all their decisions, making it difficult to determine which agent's action was responsible for a good or poor outcome.

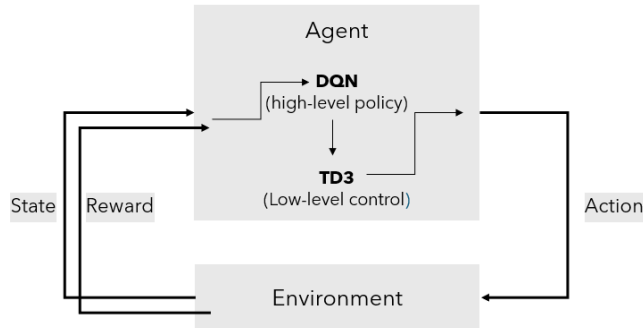


Figure 8: The structure of the Hybrid agent.

3.3. Evaluation

To assess the performance of the reinforcement learning agents, they are compared against the classical Black-Scholes-Merton (BSM) delta-hedging strategy as a benchmark. All agents are trained for 5000 episodes, after which they are evaluated on a new set of 1000 simulated price paths that are generated independently of the training data. By evaluating on unseen market scenarios, this approach tests the generalization ability of the learned policies and ensures that the results are not driven by overfitting to the training environment. These evaluation metrics are applied across three distinct market environments: a frictionless market (no transaction costs), a baseline market (1% transaction cost), and a high-friction market (5% transaction cost).

To evaluate how well the agents perform, four different aspects are considered.

- First, the mean hedging cost and standard deviation of the agents trading strategy are considered. The mean hedge cost represents the average total expense required to maintain the hedge over the life of the option, computed across 1000 independently simulated price paths. For each path, the total hedging cost is calculated by accumulating the P&L at each time step according to Equation (16), and the mean is then taken across all paths. A mean cost of zero indicates that the hedge is free to hold, while a mean cost of 100% shows that the cost of holding the hedge offsets the premium received.

The standard deviation is computed across the same 1000 simulated price paths and measures the consistency and reliability of the hedging strategy across different market scenarios. A lower standard deviation indicates a more robust hedge that is less sensitive to specific price trajectories.

The standard error of the mean (SEM) is computed and used to measure the precision of the sample mean. It estimates how far the sample average is from the true population mean. A smaller standard error of the mean indicates the sample mean is closer to the true population mean. A large standard error of the mean indicates greater uncertainty, and the sample mean may differ from the population average.

- Secondly, the policies for each agent are evaluated. The policies determine the hedging ratio each agent would use based on time to maturity and moneyness. By comparing these 3D surfaces to the theoretical BSM policy, we can identify behaviors such as over-hedging or transaction-cost awareness.
- Third, the mean hedging error is calculated. It is the expected value of the financial shortfall or surplus that occurs when replicating a derivative's payoff with a trading portfolio. To measure the replication's precision, the mean hedging error is calculated at the terminal time step T , and is defined as:

$$\text{mean hedging error} = \frac{1}{N}(V_T - H_T),$$

where $V_T = \max(S_T - K, 0)$ is the option payoff at maturity. S_T denotes the final stock price, and K is the strike price. N is the number of trials used for evaluation. H_T is the final value of the hedging portfolio, consisting of the option premium received at the start of the period when the option is sold, and the cumulative P&L generated by the hedging strategy over the life of the option, including all transaction costs. This follows the terminal portfolio value formulation introduced by BUEHLER et al. (2018, p. 6, Equation 2.1). A hedging error of zero corresponds to perfect replication. A positive hedging error indicates that the portfolio was insufficient to cover the option payoff, while a negative hedging error indicates that the portfolio exceeded the option payoff.

Additionally, the standard deviation of the hedging error is measured. It shows the dispersion of hedging errors around the mean, indicating how sensitive the agent's replication performance is to different price trajectories.

Additionally, the Root Mean Squared Error (RMSE) is calculated. It is formulated as:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - Q(s_i, a_i))^2}$$

where y_i represents the target return and $Q(s_i, a_i)$ is the agent's predicted value function. It captures the standard deviation and the mean bias of the network's function approximation errors, measuring the difference between the agent's expected return predictions and the observed bootstrapped target values.¹⁵ A perfect hedging policy would yield an RMSE of zero.

The difference between hedging error and hedging cost is that the hedging error measures how accurately the hedging portfolio replicates the option payoff at maturity, while the hedging cost reflects the overall economic cost of maintaining the hedge.

Finally, we examine specific hedging trajectories. This provides a qualitative look at how different agents respond to price fluctuations in real-time during a single simulated episode, highlighting the differences between discrete and continuous action spaces.

¹⁵See <https://www.sciencedirect.com/topics/engineering/root-mean-square-error>. Last accessed 27.05.26.

4. Results and Discussion

This section evaluates the performance of the four deep reinforcement learning (DRL) agents, DQN, DDPG, TD3, and the Hybrid model, against the classical Black-Scholes-Merton (BSM) benchmark. The analysis is conducted across three market scenarios characterized by increasing transaction costs: frictionless ($\kappa = 0$), presented in Section 4.1, realistic ($\kappa = 0.01$), presented in Section 4.2 and stressed ($\kappa = 0.05$) presented in Section 4.3. The transaction cost parameter κ is found in the last term in Equation (16): $\kappa|S_i(H_{i+1} - H_i)|$. As seen in the equation, every time the agent changes its hedge position, it pays a cost proportional to both the size of the change and the current asset price. If the agent does not re-balance, no cost is incurred.

All of the results presented in this section are based on agents trained for 5000 episodes and then evaluated on 1000 new price paths. Results for 100 and 1 000 training episodes are provided in Appendix B for reference. The parameters used throughout the results are found in Appendix A, where Table 7 specifies the environment settings and Table 8 specifies the neural network architecture. Section A.1 in appendix provides a sensitivity analysis regarding hyperparameter choices.

In each of the three scenarios, the mean cost of maintaining the hedge is discussed along with the standard deviation of the given agent and transaction cost. The policies for each agents are presented and evaluated. The policy represent what hedging ratio the agent would choose given the time to maturity and moneyness. Finally, the hedging error is presented and discussed for each given agent.

Following the scenario analysis, Section 4.5 provides a qualitative visualization of hedging trajectories on two specific price paths under the realistic market with transaction costs of 1%.

Regarding the Hybrid agent, two variants were implemented as described in Section 3.2.4: one where the DQN and TD3 are trained simultaneously, and one where they are trained sequentially. As no substantial performance difference was found between the two approaches, and the simultaneous variant required considerably less computation time, all Hybrid agents results presented in this section refer to the simultaneously trained agent. A full comparison between the two variants is provided in Appendix C.

4.1. Scenario A: The Frictionless Benchmark (No Transaction Costs)

In this section, the performance of reinforcement learning agents is evaluated in an idealized environment where transaction costs are absent ($\kappa = 0$). This scenario serves as a control group to verify the agents' ability to learn the underlying mathematical principles of delta hedging without the penalty of re-balancing costs. The results of this experiment directly address **RQ1**.

4.1.1. Cost under a Frictionless Market

In a frictionless environment, the performance of the various agents aligns closely with classical financial theory, serving as a baseline for the subsequent analysis of market frictions. As shown in Table 1, the Black-Scholes-Merton model achieves a mean cost of 0.88%, which, while not zero, confirms that discrete delta hedging remains a highly effective approximation of theoretically perfect continuous hedging. The slight deviation from zero is due to discretization error, which is a byproduct of trading at fixed time intervals (dT) rather than continuously.

All reinforcement learning agents converged to near-zero mean costs during training. The DQN agent recorded the lowest mean cost at -0.11% , followed by TD3 (-0.33%) and DDPG (0.39%). The Hybrid agent reported a slightly higher mean cost of 1.60% . The cost of holding the hedge is almost zero, meaning that it is almost free to uphold. These results demonstrate that RL agents can learn to hedge effectively, even when starting from a random state without prior knowledge of the Black-Scholes formula.

Table 1: Mean hedge cost and standard deviation under the Frictionless Market (no transaction cost).

	BSM	DQN	DDPG	TD3	Hybrid
Mean costs (% of option price)	0.88	-0.11	0.39	-0.33	1.60
Std (% of option price)	25.53	30.11	28.80	35.45	29.85
Standard error of the mean	0.81	0.95	0.91	1.12	0.94

Section 3.3 describes how the means and standard deviations found in Table 1 are conducted. Despite comparable mean costs across the models, the BSM model maintains the highest consistency, as seen by its standard deviation of 25.53%. The RL agents exhibit slightly higher volatility in their hedging performance, with standard deviations ranging from 28.80% to 35.45%. This suggests that while RL agents can match the expected outcome of the BSM model, the latter remains the most reliable strategy in the absence of transaction costs.

Figure 9 is a visualization of the results in Table 1 and we can see that the results are almost perfect in the shape to the BSM model, proving that our agents are learning the optimal hedging strategy of Black-Scholes when there is no transaction costs involved.

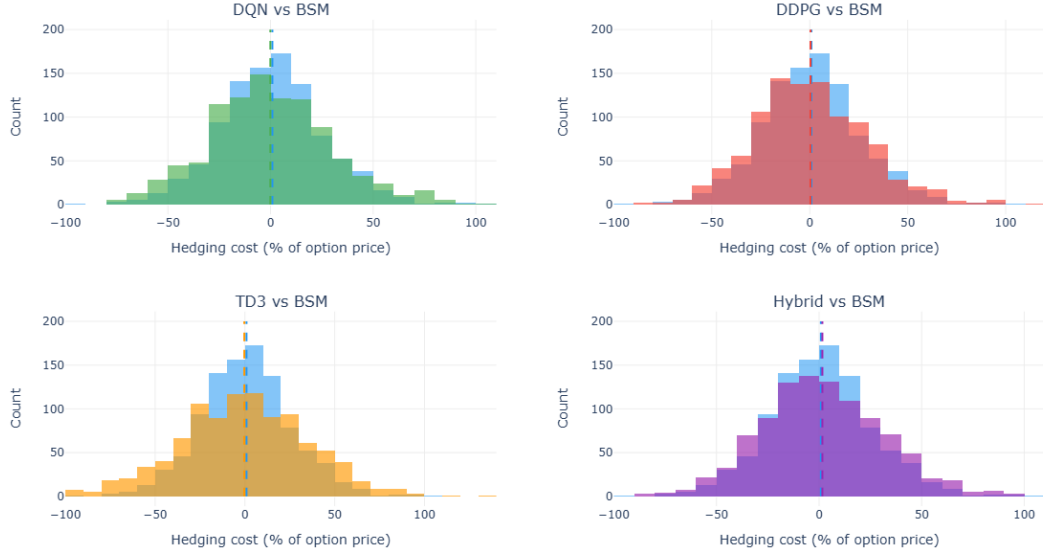


Figure 9: Histograms of the cost distribution under the Frictionless Market (no transaction costs). The blue histograms represent the BSM model, the green the DQN, the red DDPG, the yellow the TD3, and the purple the Hybrid. The vertical lines represent the mean hedge cost.

4.1.2. Policy under a Frictionless Market

Beyond the cost metrics, the learned policies offer a more direct view of what each agent has actually learned.

Figure 10 shows a visualization of the BSM policy. The hedge ratio represents what action the agent would act upon, given the time to maturity and moneyness. The colors range from dark green when the hedge ratio is one, to red when the hedge ratio is zero, and the same colors will be used throughout every representation of the hedge ratio. To clarify, the BSM model will not need multiple episodes to converge, as the other agents need, since it is a mathematical framework for calculating the theoretical fair value of a European-style option, and not an agent that would need multiple episodes to learn a strategy.

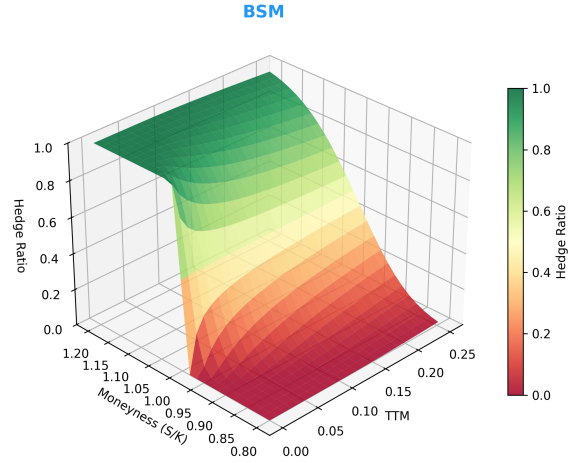


Figure 10: A visualization of the policy for the theoretical Black-Scholes-Merton model.

In Figure 10, we can see a clear pattern that for *in-the-money options* (ITM)¹⁶, the policy has a hedge ratio of one. For *out-of-the-money options* (OTM)¹⁷, the hedge ratio goes down to zero, and near *at-the-money* (ATM)¹⁸, the policy shows a steep gradient, reflecting high sensitivity to underlying price changes.

The policies obtained by the reinforcement learning agents in this frictionless environment are presented in Figure, 11 and 12 for the DDPG and TD3 agent. In the absence of transaction costs, both continuous agents almost perfectly mimic the shape of the BSM model. This high degree of alignment is due to the continuous action spaces, which enable precise adjustments necessary to replicate the theoretical benchmark.

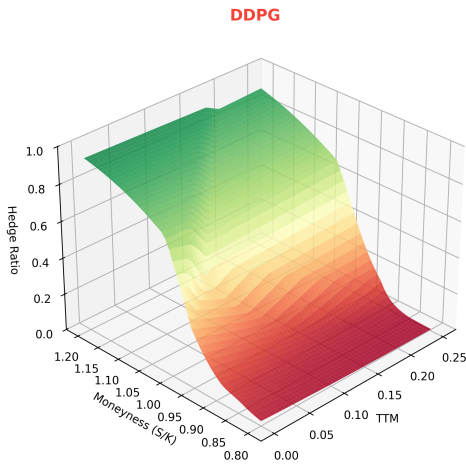


Figure 11: The optimal policy for the DDPG agent under the Frictionless Market (no transaction cost)

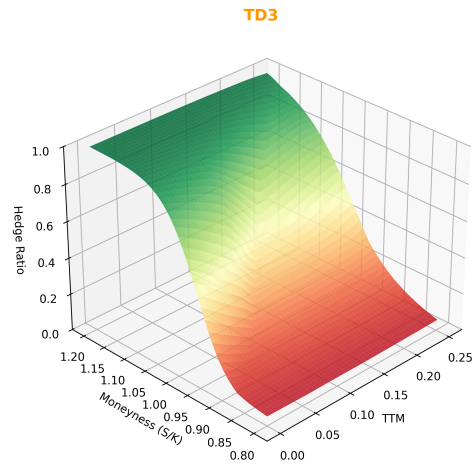


Figure 12: The optimal policy for the TD3 agent under the Frictionless Market (no transaction cost).

In contrast, discrete-action agents exhibit a different visual profile. Figure 13 shows the policy for the DQN, while Figure 14 presents the results for the Hybrid agent. Both agents display a "jagged" pattern, which is a direct consequence of the discrete action space. However, we can see that the optimal policy of the agents are one when we are ITM, zero when we are OTM, and a gradient when we are ATM, reflecting high sensitivity to price changes.

¹⁶In-the-money refers to an option with a strike price below the current market price of the underlying asset (for a call option).

¹⁷Out-of-the-money refers to options with a strike price higher than the current market price of the underlying asset (for a call option).

¹⁸At-the-money refers to an option with a strike price very close to the current market price of the underlying asset.

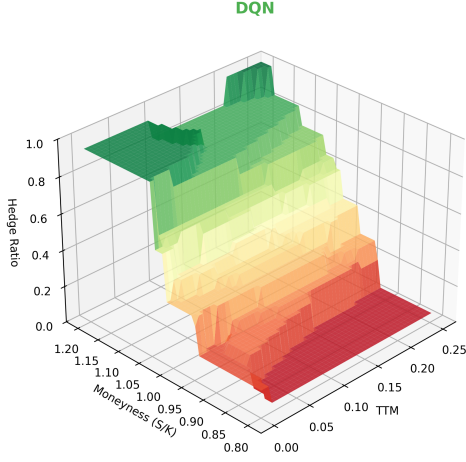


Figure 13: The optimal policy for the DQN agent under the Frictionless Market (no transaction cost).

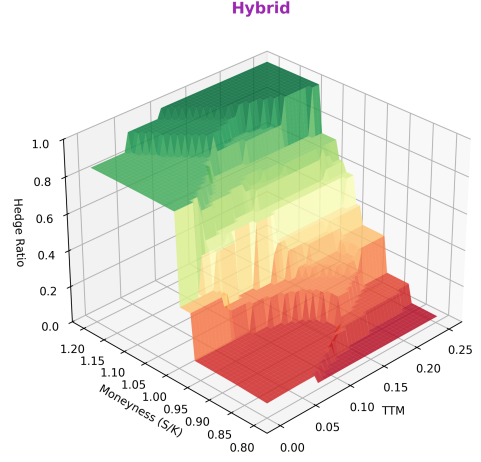


Figure 14: The optimal policy for the Hybrid agent under the Frictionless Market (no transaction cost).

The Hybrid agents policy show similar attributes to the DQNs policy. The reason for this is that the high level of bin selection is dominant over the low level of refinement from TD3. The Hybrid agent suffers from the credit assignment problem, where the TD3 agent and the DQN are collectively punished, leading the TD3 agent to "lie flat" in its decision-making to avoid negative rewards.

4.1.3. Model Evaluation under Frictionless Markets

While the analysis of average costs provides a measure of expected profitability, the hedging error, defined as the terminal difference between the option payoff (V_T) and the value of the hedging portfolio (H_T), offers a more critical look at replication precision and total risk exposure. Table 2 summarizes the hedging error for the agents in a frictionless market and the way that the results are conducted are found in Section 3.3.

Table 2: Hedging error metrics for the different agents under the Frictionless Market (no transaction cost).

	BSM	DQN	DDPG	TD3	Hybrid
Mean Hedging error	0.080	0.040	0.060	0.031	0.109
Std Hedging error	6.210	6.045	6.031	6.154	6.111
RMSE Hedging error	6.210	6.045	6.031	6.154	6.112

In classical financial theory, a continuous BSM hedge would result in an error of zero; however, the non-zero Root Mean Squared Error (RMSE) observed here is a result of discretization error, as discussed previously. The BSM can however be seen as the primary benchmark for reliability in a frictionless market.

The mean hedging error is relatively close to zero for all agents, showing that none of the strategies exhibit a strong systematic bias towards either over-hedging or under-hedging. Among the reinforcement learning agents, the TD3 agent achieved the lowest mean hedging error of 0.031, followed by the DQN agent with 0.040. These two agents were able to replicate the option payoff slightly more accurately on average compared to both the DDPG and Hybrid agents, as well as the BSM benchmark.

The standard deviation of the hedging error reflects the variability of the replication error and can therefore be interpreted as a measure of hedging risk. The DDPG agent achieved the lowest standard deviation of 6.031, closely followed by the DQN agent with 6.045. This suggests that these agents produced a slightly more stable hedging performance across different simulated market paths. In comparison, the TD3 agent achieved the lowest mean error but a somewhat higher standard deviation of 6.154, indicating that although the average replication accuracy was strong, the variability of the outcomes remained slightly larger.

The RMSE values are nearly identical to the standard deviations because the mean hedging errors are close to zero for all agents. This means that the overall hedging performance is primarily driven by the variability of the hedging error rather than by systematic bias. Overall, the results suggest that the reinforcement learning agents were capable of learning hedging strategies that perform competitively with the classical BSM benchmark under frictionless market conditions. In particular, the DQN and DDPG agents appear to provide slightly more stable hedging performance, while the TD3 agent achieved the lowest average replication error.

4.2. Scenario B: The Realistic Market (Baseline Costs)

In this section, the performance of agents is evaluated in a realistic market environment where transaction costs are set at 1% ($\kappa = 0.01$). This scenario serves as a way to evaluate how well the agents would perform in a market where the execution of trades is associated with costs. The results are used to address **RQ2** and **RQ3**, examining whether DRL agents can learn more cost-efficient hedging strategies than the BSM benchmark when market frictions are present, and how the choice of agent architecture affects the stability and reliability of the hedging performance in such conditions.

4.2.1. Cost under Realistic market

The introduction of 1% transaction costs changes the results compared to the frictionless baseline. As shown in Table 3, the BSM model is the most affected, with its mean hedging cost increasing from 0.88% to 90.80% of the option price. This means that the cost of maintaining the hedge consumes nearly the entire option premium (90.80%), leaving almost no margin for the option writer. The increase reflects the weakness of the BSM strategy in friction markets, where it readjusts regardless of cost, leading to transaction fees accumulating over the lifetime of the option.

Table 3: Mean hedge costs and standard deviations under the Realistic Market (Baseline cost)

	BSM	DQN	DDPG	TD3	Hybrid
Mean costs (% of option price)	90.80	45.11	51.85	51.47	42.91
Std (% of option price)	36.70	45.60	37.44	42.49	44.32
Standard error of the mean	1.16	1.44	1.18	1.34	1.40

All reinforcement learning agents achieve lower mean costs than the BSM benchmark, ranging from 42.91% for the Hybrid agent to 51.85% for the DDPG. The price of upholding the hedge is almost half compare to the BSM model. This suggests that the RL agents have learned to re-balance more selectively, avoiding unnecessary trades that would otherwise erode the hedging portfolio.

In terms of consistency, however, the BSM model retains an advantage. Its standard deviation of 36.70% is lower than for all RL agents, which range from 37.44% to 45.60%. This indicates that while the RL agents reduce mean costs compared to the BSM model, they do so with somewhat greater variability across different price paths. The cost distributions are visualized in Figure 15, where the wider spread of the RL agents relative to BSM is visible.

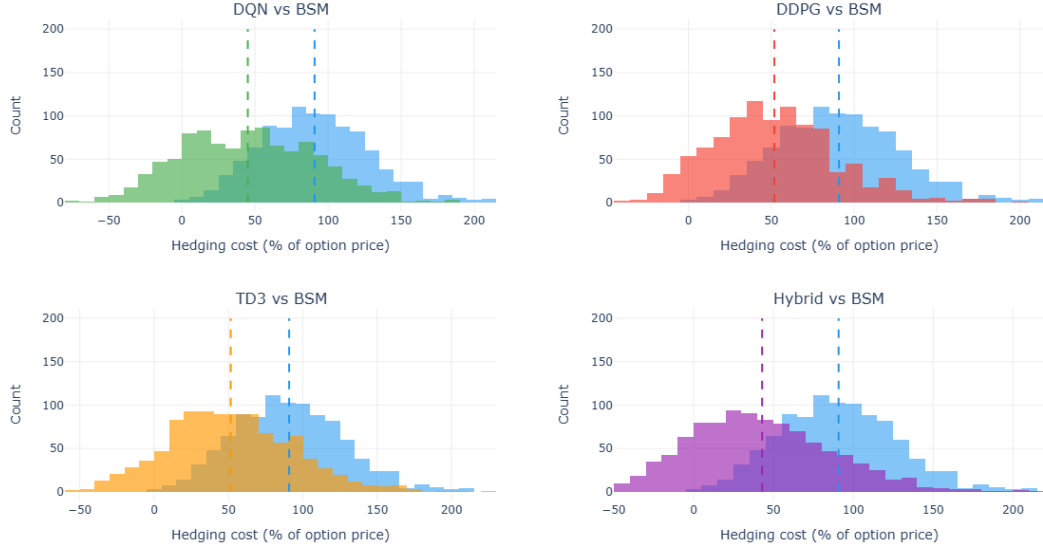


Figure 15: Histograms of the cost distribution under the Realistic Market (1% transaction costs). The blue histograms represents the BSM model, the green the DQN, the red DDPG, yellow the TD3 and the purple the Hybrid. The vertical lines represents the mean.

4.2.2. Policy under the Realistic Market

In this section, the results from the policy will be evaluated. Figure 16 show the policy for the DDPG agent, Figure 17 show the policy for the TD3 agent.

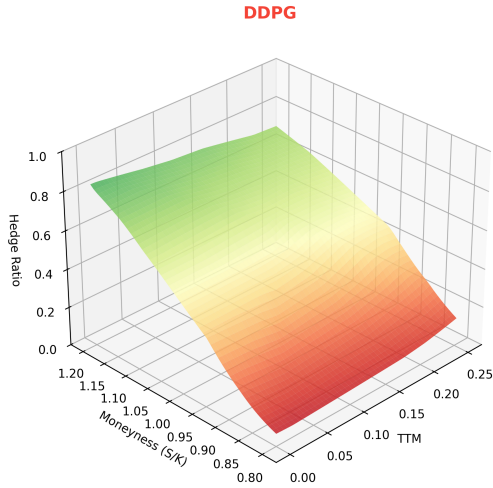


Figure 16: The optimal policy for the DDPG agent under the Realistic Market (1% transaction costs).

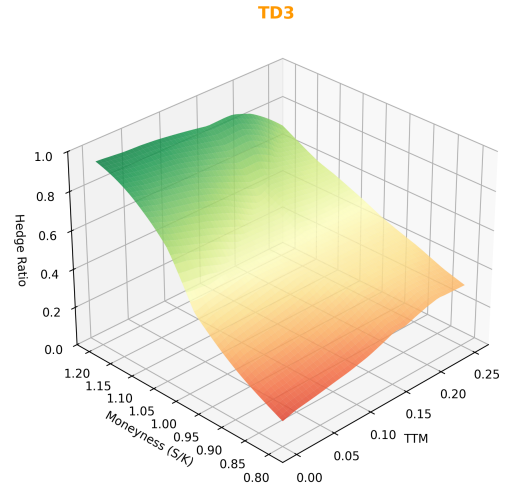


Figure 17: The optimal policy for the TD3 agent under the Realistic Market (1% transaction costs).

The continuous-action agents, DDPG and TD3, shown in Figures 16 and 17, retain the general shape of the BSM reference surface but with some changes. The transition is smoother and less steep than the theoretical delta, and their policies in frictionless markets. We can also see that the TD3 maintains a more elevated hedge ratio throughout compared to the

DDPG.

Figure 18 show the policy for the DQN agent, and Figure 19 shows the policy for the Hybrid agent.

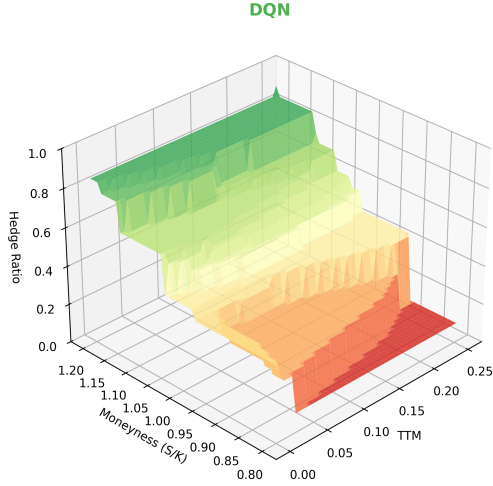


Figure 18: The optimal policy for the DQN agent under the Realistic Market (1% transaction costs).

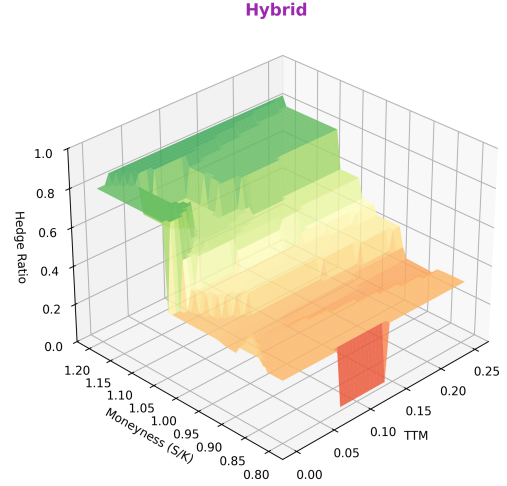


Figure 19: The optimal policy for the Hybrid agent under the Realistic Market (1% transaction costs).

The DQN agent and the Hybrid agent retain the overall structure as their policies under the frictionless market. However, the agents are grouping hedge ratios into larger buckets, so the change in moneyness needs to be large enough to justify the transaction cost. Additionally, they never reach a hedge value of 1, and they have fewer instances of 0 for OTM states. This compression of the hedge ratio range reflects the agents' cost-awareness: the transaction cost penalty discourages large positional changes, leading the agents to maintain a more stable hedge rather than tracking the theoretical delta.

4.2.3. Model Evaluation under the Realistic Market

Table 4 summarizes the hedging error metrics for the realistic market scenario, where transaction costs are 1% ($\kappa = 0.01$).

Table 4: Hedging error metrics for the different agents under the Realistic Market (1% transaction costs).

	BSM	DQN	DDPG	TD3	Hybrid
Mean Hedging Error	3.68	1.85	2.12	2.11	1.76
Std Hedging Error	6.40	6.17	6.08	6.31	6.31
RMSE Hedging Error	7.39	6.44	6.44	6.65	6.55

Under realistic market assumptions, all reinforcement learning agents perform better the

Black-Scholes benchmark in terms of bias. The BSM model shows the highest mean hedging error at 3.68. This difference occurs because the BSM model re-balances "blindly" based on theoretical delta, failing to consider the cumulative effect of transaction costs.

Among the RL models, the Hybrid agent achieves the lowest mean hedging error (1.76) followed by the DQN (1.85). This result suggests that the discrete action space of the DQN may act as a natural regularizer, preventing the agent from over-adjusting and incurring unnecessary costs that would otherwise increase the bias. The continuous-action agents, DDPG and TD3, follow closely with mean hedging values of 2.12 and 2.11, respectively. All agents received a lower bias compared to the BSM model.

In terms of standard deviation, all RL agents achieve lower values than the BSM model, indicating that they not only reduce the systematic bias but also produce more consistent results across different price paths. The DDPG achieves the lowest standard deviation at 6.08, followed by the DQN at 6.17, though the differences are small and no clear conclusion can be drawn regarding the effect of architecture on variance alone.

Unlike in the frictionless scenario, the RMSE values now diverge more from the standard deviations. This reflects the fact that the mean hedging errors are no longer close to zero, meaning the systematic bias contributes to the overall RMSE. The BSM model records the highest RMSE at 7.39, confirming that it performs worst in terms of both bias and variance when transaction costs are present.

Taken together, these results demonstrate that in the presence of 1% transaction costs, deep reinforcement learning agents learn more effective terminal replication strategies than the classical delta-hedging approach, reducing both the systematic bias and the variability of the hedging error.

4.3. Scenario C: The Stress Test (High Transaction Costs)

The final scenario evaluates the agents under high market frictions, where transaction costs are increased to 5% ($\kappa = 0.05$). This setting serves as a stress test, designed to examine how the different hedging strategies perform when trading becomes very expensive. The results are used to further address **RQ2** and **RQ3**, providing insight into both the cost efficiency and architectural differences between agents under extreme market conditions.

4.3.1. Cost for the Stress test

Table 5 presents the mean hedging cost and standard deviation for all agents under 5% transaction costs and in Figure 20 the corresponding distribution is visualized. In a market with 5% transaction costs, the primary objective of a hedging strategy shifts from eliminating risk to economic survival.

Table 5: Mean hedge costs and standard deviations under the Stressed Market (5% transaction costs).

	BSM	DQN	DDPG	TD3	Hybrid
Mean costs (% of option price)	456.66	106.85	166.23	242.00	157.34
Std (% of option price)	127.33	126.13	116.37	79.23	84.37
Standard error of the mean	4.03	3.99	3.68	2.51	2.67

The results further illustrate the limitations of continuous delta hedging in high-friction markets. The Black–Scholes benchmark readjusts the hedge regardless of trading costs, causing transaction fees to accumulate. As a result, the BSM strategy produces the highest mean hedging costs of 456.66% of the option price.

In contrast, all reinforcement learning agents are performing better than the BSM benchmark in terms of both mean hedge cost and lower standard deviation. The DQN agent achieves the lowest mean cost at 106.85%, roughly four times more efficient than BSM. The Hybrid agent follows at 157.34%, and DDPG at 166.23%. TD3 achieves a mean cost of 242.00%, which is notably higher than DQN and DDPG in this scenario, though still a lot lower than BSM.

Regarding consistency, the picture is different. TD3 achieves the lowest standard deviation at 79.23%, followed by the Hybrid agent at 84.37%, indicating the most stable and predictable outcomes across evaluation paths. The DQN agent shows the highest standard deviation among RL agents at 126.13%, implying that while it finds cost-efficient strategies on average, its performance is less consistent across different price trajectories. The results are visualized in Figure 20.

Both the mean costs and standard deviations are much higher in this scenario compared to the previous experiments, reflecting the increased uncertainty and difficulty associated with hedging under high transaction costs.



Figure 20: Histogram of the cost distribution, under the Stressed Market (5% transaction costs). The blue histograms represents the BSM model, the green the DQN, the red DDPG, yellow the TD3 and the purple the Hybrid. The vertical lines represents the mean.

4.3.2. Policy under the Stress Test

The policies that the different agents are obtaining under the market with 5% transaction costs are presented in Figure 21 for the DDPG, Figure 22 for the TD3.

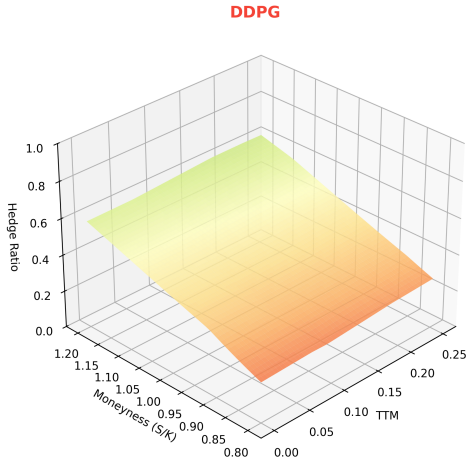


Figure 21: The optimal policy for the DDPG agent under the Stressed Market (5% transaction costs).

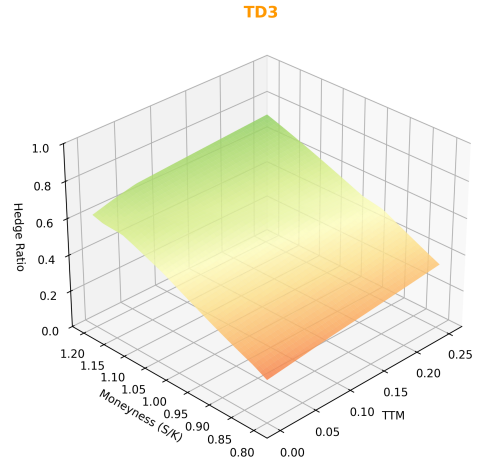


Figure 22: The optimal policy for the TD3 agent under the Stressed Market (5% transaction costs).

Both DDPG and TD3 display considerably flatter surfaces compared to the previous scenarios, though some curvature in the moneyness dimension remains, suggesting the agents still respond to market conditions to some degree.

Figure 23 shows the policy for the DQN, Figure 24 shows the policy for the Hybrid agent.

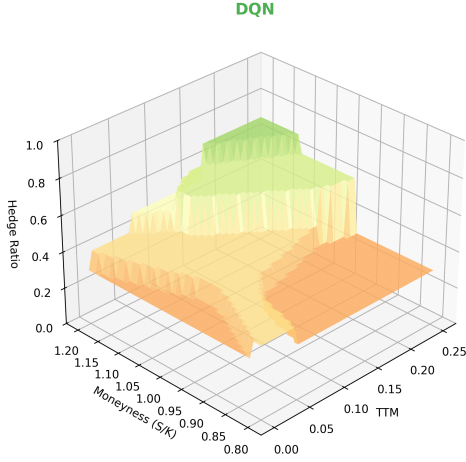


Figure 23: The optimal policy for the DQN agent under the Stressed Market (5% transaction costs).

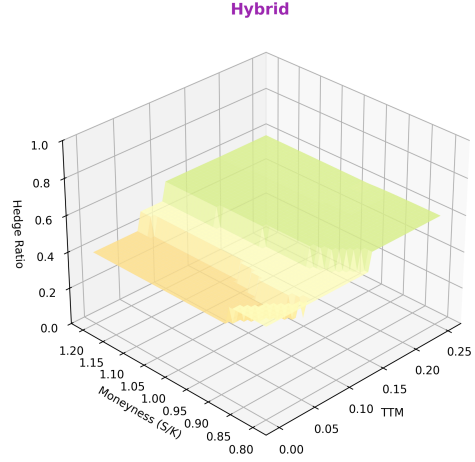


Figure 24: The optimal policy for the Hybrid agent under the Stressed Market (5% transaction costs).

The DQN and Hybrid agents have also shifted toward a flatter surface compared to the previous two scenarios. Both agents still retain the jagged pattern of their discrete action.

From the policies, it is clear that the agents settle on intermediate hedge ratios across a wide range of states, avoiding the extremes of zero and one, even for deep out-of-the-money and deep in-the-money options, respectively. This behavior reflects a rational response to high transaction costs: repositioning to the boundary hedge ratios would incur high costs, and the agent has learned that maintaining a moderate, stable position is preferable. The agents are developing "no trade zones", where the cost of trading is too high compared to the loss if the agent stays semi-hedged.

In Appendix D an additional run is presented. While all RL agents again outperform the BSM benchmark, the ranking among agents shifts, with TD3 achieving the lowest mean cost. This run-to-run variability, underscores the sensitivity of deep RL training to stochastic initialization under high market frictions.

4.3.3. Model Evaluation Under the Stressed Markets

Table 6 presents the hedging error metrics for all agents under 5% transaction costs. The results shows a big divergence between the BSM model and the RL agents in terms of terminal replication precision.

Table 6: Hedging error metrics for the different agents under the Stressed Market (5% transaction costs).

	BSM	DQN	DDPG	TD3	Hybrid
Mean hedging error	19.04	5.03	7.41	10.4	7.06
Std hedging error	8.56	8.06	7.95	7.24	7.47
RMSE hedging error	20.88	9.51	10.87	12.71	10.28

The BSM model performs worst, with a mean hedging error of 19.04. This large positive mean error indicates that the BSM portfolio systematically fails to replicate the option payoff at maturity.

All RL agents achieve lower hedging errors; the best performer, DQN, records a mean error of 5.03. TD3, despite its higher mean cost, achieves the lowest hedging-error standard deviation of 7.24. The standard deviation of the BSM model isn't too large compared to the other agents. All agents exhibit both smaller bias and variance compared to the BSM model, demonstrating that they are more reliable in a stressed market.

In summary the stress test results confirms that reinforcement learning agents are capable of adapting their strategies to extreme market frictions in ways that the classical Black–Scholes approach cannot. Unlike the BSM model, which re-balances regardless of cost, RL agents learned to readjust the hedge less frequently.

4.4. Discussion Around the Different Scenarios

Three market scenarios have now been presented and evaluated, providing information on how reinforcement learning agents respond to varying levels of transaction costs. In the frictionless scenario, all agents converged toward a policy consistent with the theoretical BSM delta hedging strategy. This serves as an important validation, demonstrating that the agents are capable of discovering the theoretically optimal solution when no market frictions are present, thereby confirming that the learning framework is correctly implemented.

As transaction costs are introduced, the behavior of the agents changes. Rather than continuously tracking the theoretical delta prescribed by the BSM model, the reinforcement learning agents learn to readjust the hedge less frequently, accepting some hedging imprecision in exchange for lower trading costs. The agents seems to develop a "safe" policy and not re-balance as frequent under a stressed market. To deepen the analysis, the following two sections examine the learning curves and the run-to-run variability observed during training.

Learning Curves

One way to visualize the learning progress of an agent is through its learning curve, which illustrates how performance evolves during training and provides insights into both the speed of learning and the stability of the training process. This section presents the learning curves for the DQN agent across the three market scenarios, which are representative of the general patterns observed across all agents.

In the frictionless environment ($\kappa = 0$), the optimization problem is simplified as the system does not penalize frequent re-balancing. Consequently, the agents face a less punishing reward structure, typically achieving stabilization in the reward signal after approximately 500 episodes as visualized in Figure 25.

However, the complexity of the problem increases substantially with the introduction of transaction costs. In the realistic market scenario ($\kappa = 0.01$), the environment becomes more restrictive. As illustrated in Figure 26, the DQN agent exhibits greater instability during the initial phases of training but eventually converges after roughly 2,000 episodes.

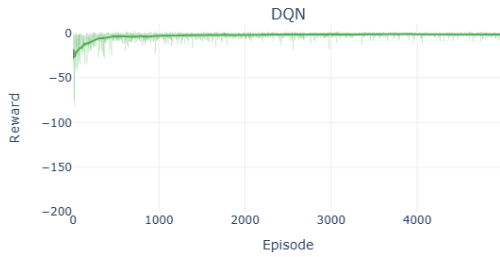


Figure 25: The learning curve for DQN agent under the Frictionless Market (no transaction cost).

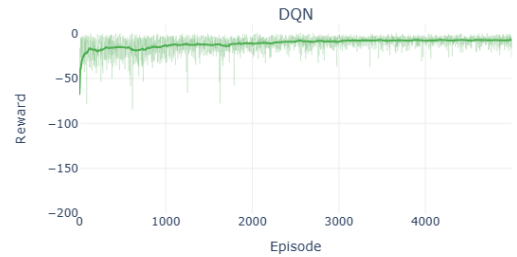


Figure 26: The learning curve for DQN agent under the Realistic Market (1% transaction costs).

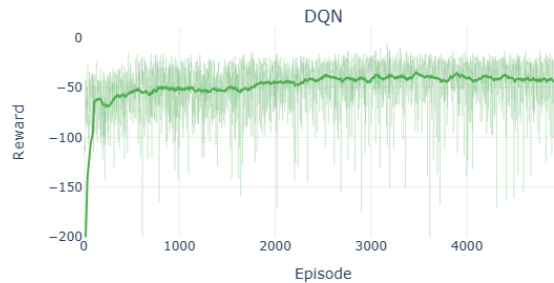


Figure 27: The learning curve for DQN agent under the Stressed Market (5% transaction costs).

The stressed market scenario ($\kappa = 0.05$) represents the most challenging environment, characterized by a highly punishing reward function. This is reflected in Figure 27, where the training process for the DQN agent remains noticeably unstable. However, this is not due to the agent failing to converge. Initially, the agent is trying out different strategies, which is reflected in the initial rise in the reward function. However, it quickly learns that trading is not an option in a high-cost market and starts practicing a semi-hedged policy. The large variance band instead reflects the randomness of the data.

Run to Run Difference

A challenge in deep reinforcement learning is instability during training. An observation made is that the same agent can perform very differently across different training runs. This variability is illustrated in Figure 28, which compares the learning curves of the TD3 agent for two separate runs trained under the same settings.

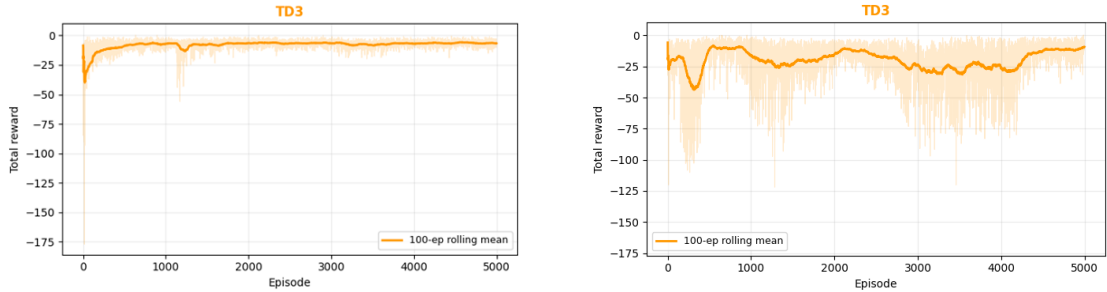


Figure 28: The learning curves for two different runs for the TD3 agent under the Realistic Market (1% transaction costs).

When looking at Figure 28, we can see that something interesting has happened to the TD3 agent, which appears highly unstable in the training process to the right. This is contrary to what the theory in Section 3.2.3 would suggest, since TD3 incorporates mechanisms specifically designed to stabilize training. The reason for this result might be due to the sensitivity of deep reinforcement learning algorithms to random initialization and the stochastic nature of training: the same algorithm can produce very different learning trajectories across runs depending on initialization of the neural network weights, random sampling from the replay buffer, and the OU noise which is generated. The left run, shown in Figure 28, confirms this interpretation, in this run, the TD3 agent behaves as expected, with relatively low variance and stable convergence. This run-to-run variability shows the importance of evaluating agents across multiple runs rather than relying on a single run.

4.5. Hedging Trajectory

In this section two different hedging trajectories are shown, to visualize how different agents and the Black-Scholes delta hedging approach can behave on one simulated episode. Figure 29 shows what the hedge trajectory can look like if the underlying assets price goes up and Figure 30 shows what it can look like if the price goes down. Transaction cost is set to 1%.

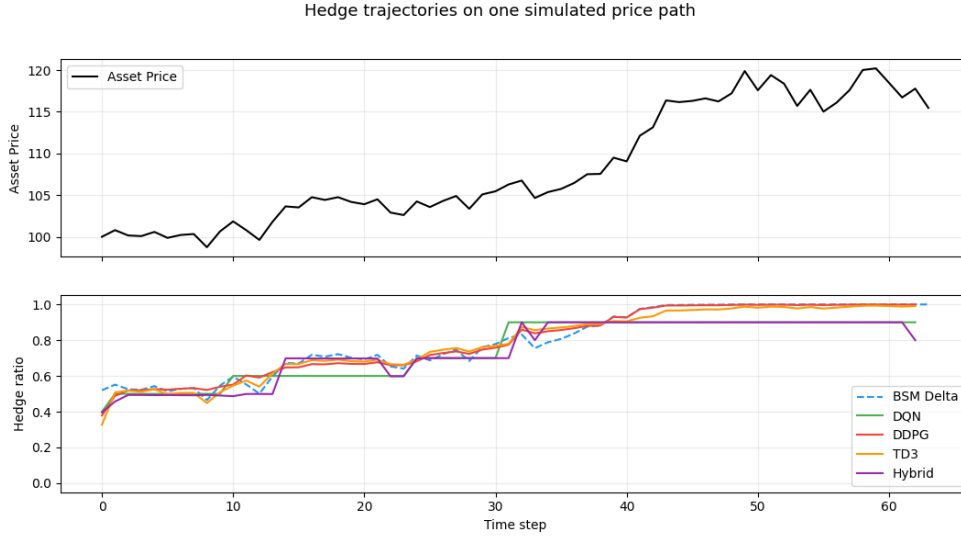


Figure 29: Hedge trajectories for the agents and BSM model when the underlying asset goes up.



Figure 30: Hedge trajectories for the agents and BSM model when the underlying asset goes down.

Recall that the agents are hedging a short call position. When the underlying asset price rises, the option moves deeper in the money, increasing the risk exposure of the short call writer. To hedge this growing exposure, the agent must hold more of the underlying asset, hence the hedge ratio increases toward one as seen in Figure 29. Conversely, when the underlying price decreases, as shown in Figure 30, the option moves out of the money, reducing

the exposure of the short call position. In this case, the optimal response is to decrease the hedge ratio, which the agents and the BSM model do.

The hedge trajectories reflect the same architectural differences observed in the policy analysis. The DDPG and TD3 agents produce the smoothest hedge paths, which is due to their continuous action spaces. In contrast, the DQN agent displays more abrupt changes in the hedge ratio due to its discretized action space. The Hybrid agent resembles the DQN trajectory more than the TD3, suggesting that the DQN component dominates the action selection in this setting. As discussed before, this is likely a consequence of the credit assignment problem, where the TD3 agent has learned to avoid adjustments to minimize the risk of negative feedback.

5. Conclusion

This section finalizes the thesis with a summary in Section 5.1, followed by suggestions for future research in Section 5.2.

5.1. Summary

This thesis investigated the performance of four different deep reinforcement learning algorithms for option hedging across varying market scenarios. The primary objective was to evaluate and extend the methodology proposed by CAO et al. (2020) which utilizes the DDPG algorithm to derive optimal hedging strategies under transaction costs. This study expanded that framework by implementing and comparing DQN, TD3, and a hierarchical Hybrid approach.

Three different market scenarios were conducted. The first experiment was without any transaction costs and served as a baseline to investigate if agents could learn the optimal Black-Scholes Delta hedging strategy. The second experiment that was conducted involved transaction costs of 1%, and the third experiment involved a stress testing scenario with transaction costs of 5%.

The experiments were designed to answer the following research questions:

RQ1 Do deep reinforcement learning agents converge to the Black-Scholes-Merton delta hedging strategy when trained in a frictionless market environment?

RQ2 Can deep reinforcement learning agents learn hedging strategies that performs better than the Black-Scholes-Merton delta hedging strategy in markets with transaction costs?

RQ3 How does the choice of agent architecture affect the stability and reliability of the hedging performance in non-perfect markets?

The results demonstrate that all deep reinforcement agents successfully converged to the underlying mathematical principles of delta hedging in a frictionless environment, effectively mimicking the theoretical BSM policy (**RQ1**). When transaction costs were introduced, the agents adapted by learning more cost-efficient strategies, in terms of mean hedging cost, that performed better the classical BSM benchmark (**RQ2**). Regarding the last research question (**RQ3**), the results suggest that no single agent architecture consistently outperforms the others in terms of both stability and reliability across the different market scenarios. Rather, the relative strengths of each architecture depend on the level of transaction costs. Under high friction, discrete-action architectures such as the DQN achieve lower mean costs by naturally limiting re-balancing frequency, while continuous-action architectures such as TD3 produce more stable and predictable outcomes. Additionally, the stochastic nature of deep RL training introduces run-to-run variability that is independent of architecture choice, further complicating any definitive conclusion.

5.2. Future Research

While our results show that the potential of deep reinforcement learning algorithms used for hedging, there are several areas for future research to consider.

- **Application to real market data:** Future research might focus on applying the proposed framework to real market data. This would expose the agents to market complexities not captured by simulated data, such as stochastic volatility, price jumps, and liquidity constraints.
- **Multi-agent systems:** The hybrid approach in this thesis suffers from the *credit assignment problem*. The model showed good potential in the stressed market scenarios, but the TD3 agent suffered from the shared reward system that led to the agent not taking any bug changes. The TD3 agent were also only allowed to affect the DQNs discrete choice with maximum 10%. Future research could explore individual reward signals for each component, or alternative hierarchical architectures where the credit assignment problem is explicitly addressed, potentially allowing the TD3 agent to contribute more meaningfully to the final hedging decision.
- **Multi asset portfolios:** This work has only surrounded one option. Extending the framework to manage the risk of a multi-asset portfolio could provide a broader generalization and a more sophisticated risk management.
- **More training:** The agents learning curves were instable under the stressed market scenario and they could all benefit from training from more episodes. This is often a challenge in reinforcement learning, which requires a large amount of interaction data to learn effectively.
- **Incorporation of stochastic volatility and jumps:** This work used simulated data following Geometric Brownian Motion, which assumes constant volatility. Future research could expose agents to more complex stochastic volatility models (e.g, the Heston model) or jump-diffusion process to simulate "black swan" events and fat-tail distributions found in real markets.
- **Flexible re-balancing strategies:** The trading horizon in this thesis is fixed at three months with exactly one opportunity to readjust the hedge per day, which restricts the agents from learning more flexible strategies that adapt both the timing and frequency of trades. Future research could explore different trading horizons and re-balancing frequencies, allowing agents to learn not only how much to hedge but also when it is optimal to do so.

Beyond these directions for future work, it is worth noting that deep reinforcement learning itself presents challenges that are independent of the specific setup considered here. Training is computationally demanding, sensitive to hyperparameter choices, and subject to run-to-run variability due to the stochastic nature of the optimization process. Furthermore, the learned policies are difficult to interpret, as the internal representations of neural networks offer little

transparency into why a particular action is chosen in a given state, a limitation that may be of particular concern in regulated financial environments where model explainability is required.

References

- ALBRECHT, STEFANO V., FILIPPOS CHRISTIANOS and LUKAS SCHÄFER (2024). *Multi-Agent Reinforcement Learning: Foundations and Modern Approaches*. MIT Press. URL: <https://www.marl-book.com>.
- BODIE, ZVI, ALEX KANE and ALAN J. MARCUS (2011). *Investments*. Ninth. McGraw-Hill/Irwin. ISBN: 978-0-07-353070-3.
- BUEHLER, HANS, LUKAS GONON, JOSEF TEICHMANN and BEN WOOD (2018). “Deep Hedging”. In: *arXiv preprint arXiv:1802.03042*. version 1, February 2018.
- CAO, JAY, JACKY CHEN, JOHN HULL and ZISSIS POULOS (2020). “Deep Hedging of Derivatives Using Reinforcement Learning”. In: *Journal of Financial Data Science* 3.1, pp. 10–27.
- FRANÇOIS-LAVET, VINCENT, PETER HENDERSON, RIASHAT ISLAM, MARC G. BELLEMARE and JOELLE PINEAU (2018). “An Introduction to Deep Reinforcement Learning”. In: *Foundations and Trends in Machine Learning* 11.3–4, pp. 219–354. DOI: [10.1561/22000000071](https://doi.org/10.1561/22000000071).
- FUJIMOTO, SCOTT, HERKE van HOOF and DAVID MEGER (2018). “Addressing Function Approximation Error in Actor-Critic Methods”. In: *arXiv preprint arXiv:1802.09477*.
- GÉRON, AURÉLIEN (2017). *Hands-on machine learning with Scikit-Learn and TensorFlow : concepts, tools, and techniques for building intelligent systems*. eng. Beijing: O’Reilly. ISBN: 9781491962299.
- HE, XIAOYI, DANGGUI CHEN, ZHENSHUO ZHANG and ZIMENG BAI (2025). “Hybrid DQN-TD3 Reinforcement Learning for Autonomous Navigation in Dynamic Environments”. In: *arXiv preprint arXiv:2510.26646*.
- HUANG, ERIC and YURI LAWRYSHYN (2025). “Deep Hedging Under Market Frictions: A Comparison of DRL Models for Options Hedging with Impact and Transaction Costs”. In: *Journal of Risk and Financial Management* 18.9. ISSN: 1911-8074. DOI: [10.3390/jrfm18090497](https://doi.org/10.3390/jrfm18090497). URL: <https://www.mdpi.com/1911-8074/18/9/497>.
- KINGMA, DIEDERIK P. and JIMMY BA (2015). “Adam: A Method for Stochastic Optimization”. In: *arXiv preprint arXiv:1412.6980*.
- KOLM, PETTER N. and GORDON RITTER (2019). “Dynamic Replication and Hedging: A Reinforcement Learning Approach”. In: *Journal of Financial Data Science* 1.4, pp. 159–171.
- LAPAN, MAXIM. (2018). *Deep Reinforcement Learning Hands-On*. First. Packt Publishing. ISBN: 978-1-78883-424-7.
- LILICRAP, TIMOTHY P., JONATHAN J. HUNT, ALEXANDER PRITZEL, NICOLAS HEES, TOM EREZ, YUVAL TASSA, DAVID SILVER and DAAN WIERSTRA (2015). “Continuous Control with Deep Reinforcement Learning”. In: *arXiv preprint arXiv:1509.02971*. version 6, July 2019.
- MESSAOUD, SEIFEDDINE, ABBAS BRADAI, SYED HASHIM RAZA BUKHARI, PHAM TRAN ANH QUANG, OLFA BEN AHMED and MOHAMED ATRI (2020). “A Survey on Machine Learning in Internet of Things: Algorithms, Strategies, and Applications”. In: *Internet of Things* 12, p. 100314.
- MNIH, VOLODYMYR, KORAY KAVUKCUOGLU, DAVID SILVER, ALEX GRAVES, IOANNIS ANTONOGLU, DAAN WIERSTRA and MARTIN RIEDMILLER (2013). “Playing Atari with Deep Reinforcement Learning”. eng. In: *arXiv.org*. ISSN: 2331-8422.
- SAMI, UMOR (2020). “Discrete Hedging and Pricing of European Options using Reinforcement Learning”. Accessed: 2026-05-14. MSc Thesis. Imperial College London. URL: https://www.imperial.ac.uk/media/imperial-college/faculty-of-natural-sciences/departement-of-mathematics/math-finance/SAMI_UMOR_00934743.pdf.
- SILVER, DAVID et al. (2016). “Mastering the game of Go with deep neural networks and tree search”. In: *Nature* 529.7587, pp. 484–489. DOI: [10.1038/nature16961](https://doi.org/10.1038/nature16961). URL: <https://doi.org/10.1038/nature16961>.

- SUMIEA, EBRAHIM HAMID, SAID JADID ABDULKADIR, HITHAM SEDDIG ALHUSSIAN, SAFWAN MAHMOOD AL-SELWI, ALAWI ALQUSHAIBI, MOHAMMED GAMAL RAGAB and SULIMAN MOHAMED FATI (2024). “Deep deterministic policy gradient algorithm: A systematic review”. In: *Heliyon* 10.9, e30697. ISSN: 2405-8440.
- SUTTON, RICHARD S. and ANDREW G. BARTO (2018). *Reinforcement Learning: An Introduction*. Second. Adaptive Computation and Machine Learning. Cambridge, Massachusetts, USA: The MIT Press. ISBN: 978-0262039246.
- WATKINS, CHRISTOPHER (Jan. 1989). “Learning From Delayed Rewards”. In.
- WATKINS, CHRISTOPHER and PETER DAYAN (May 1992). “Technical Note: Q-Learning”. In: *Machine Learning* 8, pp. 279–292. DOI: [10.1007/BF00992698](https://doi.org/10.1007/BF00992698).

A. Parameters

Table 7: Environment parameters

Parameters	Value
Spot price (S_0)	100
Strike price (K)	100
Maturity (T)	63
Volatility (σ)	0.2
Drift (μ)	0.05
Time step (Δt)	0.004
Risk aversion (c)	1.5
Initial position	0
Interest rate (r)	0

Table 8: Neural network parameters

Parameters	Value
Layers	3
τ	$5e^{-4}$
Discount factor (γ)	0.9995
Learn rate	$1e^{-4}$
Batch size	64

A.1. Discussion around the Hyperparameters

In this section, the hyperparameters for the agents are discussed.

Replay Buffer

The replay buffer size used for all agents throughout training was set to 1 million. Previous research by HUANG and LAWRYSHYN (2025, p. 12) suggests that a larger replay buffer can improve the learning performance of the agent by retaining informative experiences for a longer period of time. In addition, ALBRECHT et al. (2024, p. 191) argue that a replay buffer that is too small may cause valuable experiences corresponding to successful or “smart” actions to be overwritten too quickly, which can negatively affect the stability and efficiency

of the learning process. Taking this into consideration we choose to set the replay buffer size for all agents throughout training to 1 million.

Batch Size

Another important hyperparameter is the batch size, which determines the number of samples used for each training update. The setting used throughout our training was 64. Research by CAO et al. (2020, p. 10) shows that batching state-action pairs is a standard technique used specifically to stabilize the reinforcement learning process. According to ALBRECHT et al. (2024, p. 172), larger batch sizes generally lead to smoother and more stable gradient updates, since the gradients are averaged over a larger set of collected experiences. This can improve the stability of the training process, although larger batch sizes may also increase computational cost and training time.

Discount Factor

The discount factor $\gamma \in (0, 1]$ determines how much importance the agent assigns to future rewards relative to immediate ones. As noted by SAMI (2020, p. 9), a value close to one encourages the agent to consider long-term consequences, while a value close to zero leads to a more short-sighted policy focused on immediate gains. In the context of option hedging, a γ close to one is preferable, as the objective is to minimize cumulative hedging costs over the entire lifetime of the option rather than optimizing a single time step in isolation. For this reason, γ is set to 0.9995 in this thesis.

Soft Update Parameter

The soft update parameter τ controls how quickly the target networks are updated towards the main network parameters during training, see Equation (18). As noted by LILICRAP et al. (2015, p. 4), constraining the target values to change slowly improves the stability of learning. A value of τ close to zero results in slow, gradual updates, which improves training stability at the cost of slower convergence. A value closer to one causes the target network to track the main network more closely, which can accelerate learning but risks instability. The choice of τ therefore represents a trade-off between stability and learning speed. The setting used in this thesis is $\tau = 0.0005$, reflecting a preference for stability given the stochastic nature of financial markets.

Hyperparameters for the DQN agent

The DQN agent uses an epsilon-greedy exploration strategy, where the agent balances exploration and exploitation during training. As explained by CAO et al. (2020, p. 9) the agent will sometimes choose random actions in order to explore the environment, while at other times it will exploit the best action currently known according to the learned policy. The exploration behavior is controlled by the epsilon hyperparameters, which determine how much the agent explores and how quickly it transitions towards exploitation.

The epsilon value initially starts at 1.0, meaning that the agent begins training with full exploration and selects actions randomly. This is visible during training where the learning is very unstable initially. During training, the epsilon value gradually decreases according to the epsilon decay parameter. The epsilon decay parameter is in this setup 0.995. If a smaller decay value, such as 0.990, were used, the exploration rate would decrease more quickly, increasing the risk that the agent converges prematurely to a suboptimal policy, for example a nearly static hedge strategy. On the other hand, a larger decay value, such as 0.9999, would result in a slower reduction of exploration, causing the agent to continue exploring for a longer period of time.

The epsilon minimum parameter determines the lowest exploration rate allowed during training. In this setup a epsilon min of 0.01 is used, the agent will continue to choose random actions approximately 1% of the time, even after convergence. This ensures that a small degree of exploration is maintained throughout the training process. If the epsilon minimum were set to 0.00, the agent would stop exploring completely and would always follow the learned policy.

Ornstein–Uhlenbeck Noise

To encourage exploration for the agents with continuous action space, Ornstein–Uhlenbeck (OU) noise was added to the actions during training. Unlike independent Gaussian noise, OU noise is temporally correlated as seen in Equation (20). By introducing correlated exploration, the agents are encouraged to explore with temporal consistency in their hedge adjustments, which is desirable in financial hedging settings where portfolio positions typically evolve gradually over time rather than changing abruptly between consecutive time steps.

B. Training the Agents for 100 and 1000 Episodes

It is well known that reinforcement learning agents need a lot of training to perform well. To illustrate this, Tables 9 and 10 present the performance of the agents after 100 and 1000 training episodes respectively, both under 1% transaction costs, and the corresponding learning curves are shown in Figures 31 and 32.

Table 9: Mean hedge costs and standard deviations under the Realistic Market (1% transaction costs) after 100 training episodes.

	BSM	DQN	DDPG	TD3	Hybrid
Mean costs (% of option price)	90.49	41.28	34.35	38.78	40.17
Std (% of option price)	34.39	80.31	74.81	65.31	74.31
Standard error of the mean	1.09	2.54	2.37	2.07	2.35

Table 10: Mean hedge costs and standard deviations under the Realistic Market (1% transaction costs) after 1000 training episodes.

	BSM	DQN	DDPG	TD3	Hybrid
Mean costs (% of option price)	89.03	37.15	43.59	55.91	38.81
Std (% of option price)	33.76	54.68	51.69	39.57	59.70
Standard error of the mean	1.07	1.73	1.63	1.25	1.89

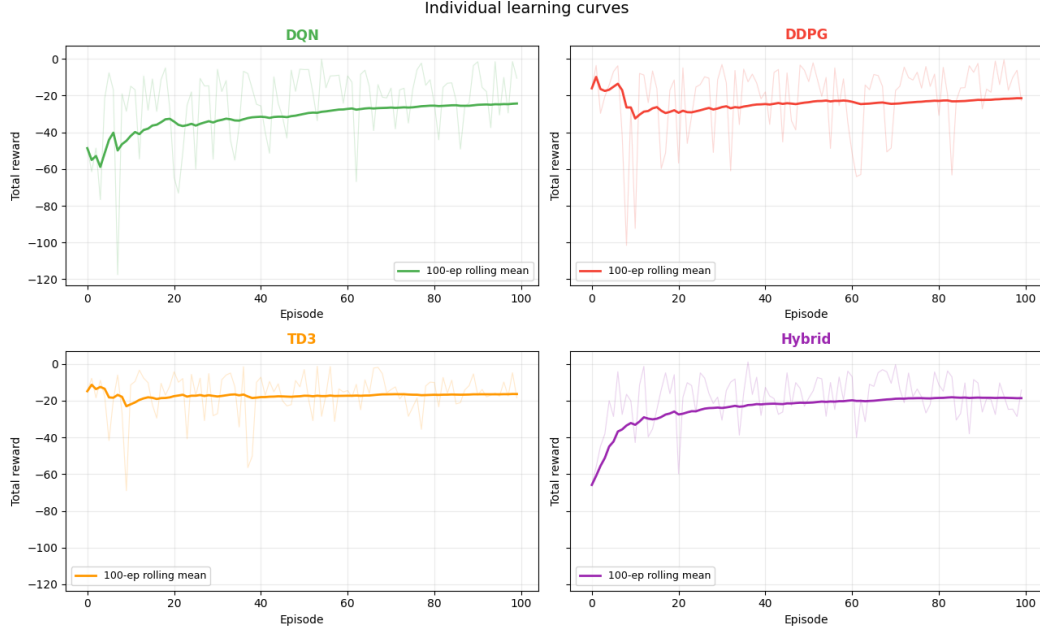


Figure 31: The learning curves for the four deep reinforcement learning agents under the Realistic Market (1% transaction costs), where the training episodes are set to a 100.

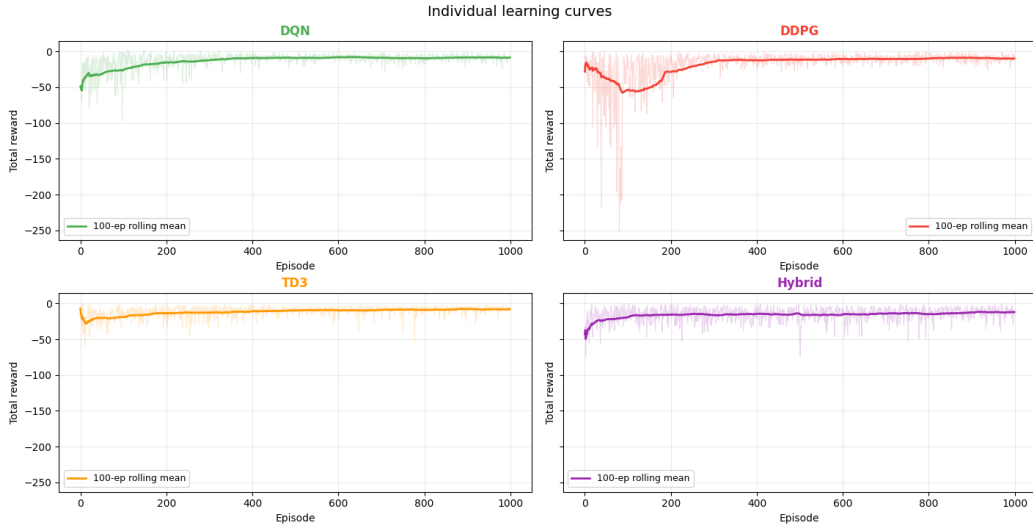


Figure 32: The learning curves for the four deep reinforcement learning agents under the Realistic Market (1% transaction costs), where the training episodes are set to a 1000.

After only 100 episodes the agents show high standard deviations, indicating that the policies

have not yet stabilized. At 1000 episodes the standard deviations have decreased across all agents, reflecting more consistent behavior, but the learning curves in Figure 32 show that the agents have still not fully converged. This progression motivates the choice of 5000 episodes as the primary training length in the main results section.

C. Hybrid Agent; Comparing Two Different Approaches

The Hybrid agent uses both the DQN and TD3 agents for action selection, where we have tried two different approaches for the Hybrid agent as described in Section 3.2.4. In Table 11 the performance of the two different Hybrid agent approaches is shown, with transaction costs set to 1%.

Table 11: Mean hedge costs and standard deviations under the Realistic Market (1% transaction costs) for the two different Hybrid agents.

	BSM	Hybrid Sequential	Hybrid Simultaneously
Mean costs (% of option price)	90.5	47.9	50.0
Std(% of option price)	35.8	52.5	52.8
Standard error of the mean	1.13	1.66	1.67

The results are similar across both variants, with the sequential approach achieving a slightly lower mean cost and standard deviation. However, the difference is small and the sequential approach requires considerably more computation time, which is why the simultaneous variant is used in the main results section. The learning curves in Figure 33 further illustrate this.

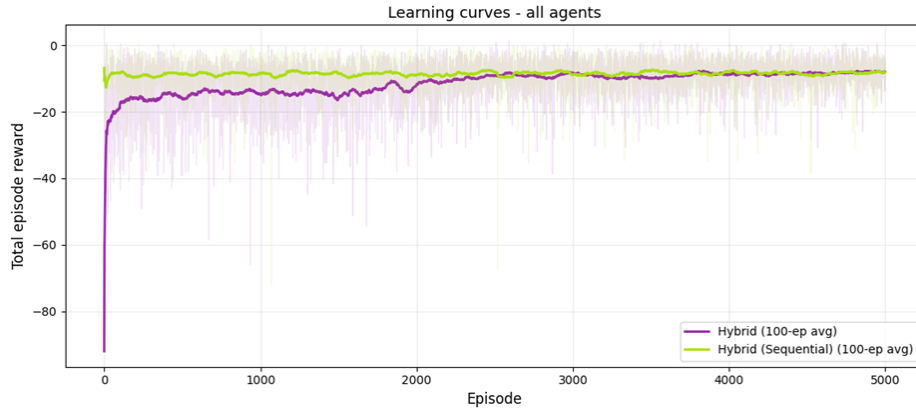


Figure 33: The learning curves for the two different Hybrid agents under the Realistic Market (1% transaction costs).

The initial gap between the two curves is misleading, in the sequential approach, the DQN is already pre-trained when the TD3 begins fine-tuning, meaning the two curves do not start from the same point. In the end, both approaches converge to similar performance levels.

D. Additional Results From the Stress Test

In this section additional results from a second run under 5% transaction costs are presented. Table 12 shows the mean hedge cost and standard deviation as a percentage of the option price and in Figure 34 the corresponding cost distributions are found. The policies can be found in Figures 35- 38 and the hedging error metrics are found in Table 13.

Table 12: Mean hedge costs and standard deviations, under the stressed market, run 2.

	BSM	DQN	DDPG	TD3	Hybrid
Mean costs (% of option price)	450.47	126.68	11.79	23.08	173.22
Std (% of option price)	132.53	85.86	168.87	152.39	79.22
Standard error of the mean	4.19	2.71	5.34	4.82	2.51

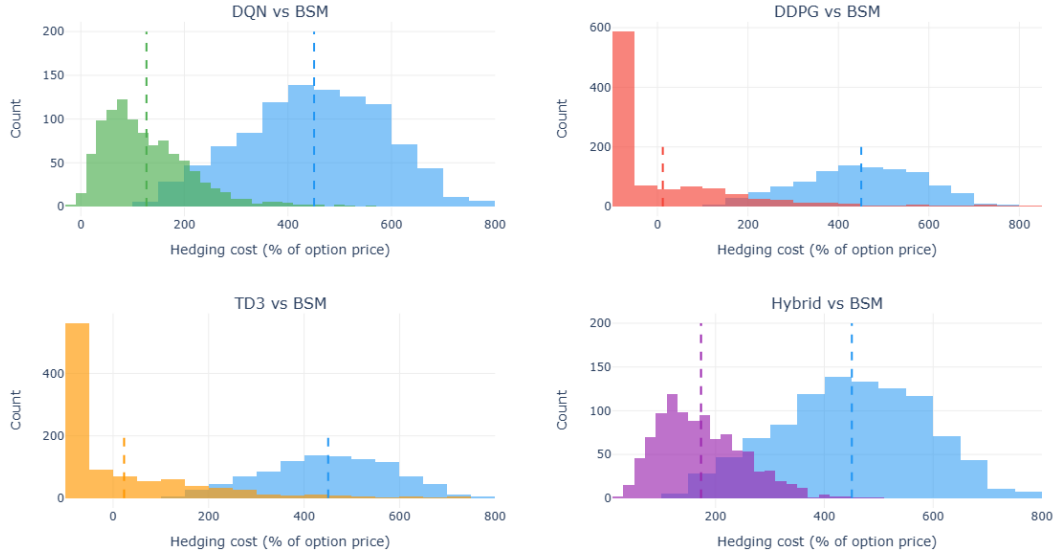


Figure 34: Histograms of the cost distribution, under the stressed market, run 2. The blue histograms represents the BSM model, the green the DQN, the red DDPG, yellow the TD3 and the purple the Hybrid. The vertical lines represents the mean.

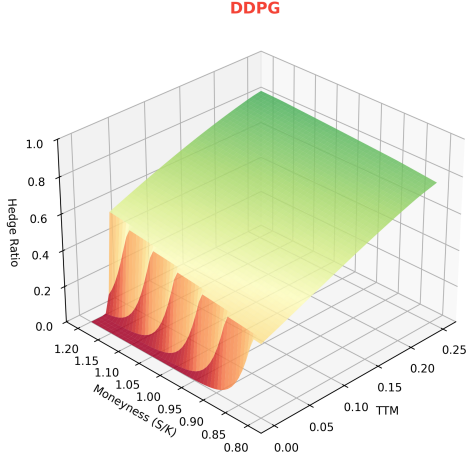


Figure 35: The optimal policy for the DDPG agent during Stress Testing, run 2.

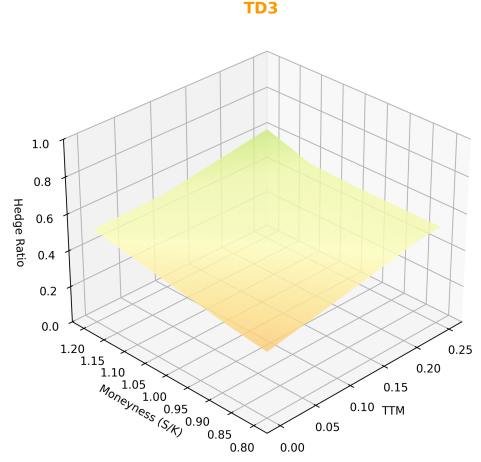


Figure 36: The optimal policy for the TD3 agent during Stress Testing, run 2.

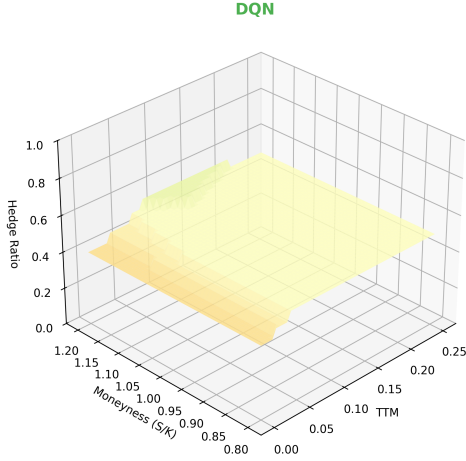


Figure 37: The optimal policy for the DQN agent during Stress Testing, run 2.

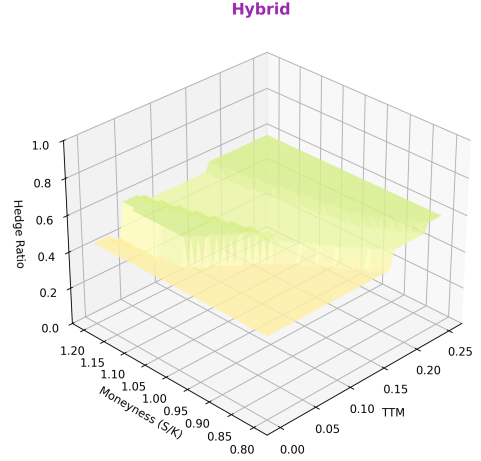


Figure 38: The optimal policy for the Hybrid agent during Stress Testing, run 2.

Table 13: Hedging error metrics for the different agents under the Stressed Market (5% transaction costs), run 2.

	BSM	DQN	DDPG	TD3	Hybrid
Mean hedging error	18.080	5.116	0.517	0.969	6.979
Std hedging error	8.504	6.732	8.987	8.644	6.923
RMSE hedging error	19.980	8.455	9.002	8.698	9.831

The most compelling result is presented in Table 12, where DDPG and TD3 achieve mean costs of 11.790% and 23.083%, respectively. While these values are substantially lower than in the first run, they are accompanied by very large standard deviations, suggesting that the agents achieve low average costs through unstable rather than consistently optimal policies. This interpretation is further supported by the DDPG policy surface shown in Figure 35,

where the hedge ratio declines sharply as TTM approaches zero, indicating that the agent increasingly reduces hedging activity close to maturity in order to avoid costly re-balancing.

Table 13, further shows that the DDPG and TD3 achieve the lowest mean hedging errors. However, these results are again associated with high standard deviations, indicating significant variability across evaluation paths rather than consistent replication performance. In contrast, the DQN and Hybrid agents exhibit more stable behavior in the second run, but they have higher average hedging costs.

E. Code

The code developed for this thesis is publicly available at the following GitHub repository: https://github.com/FeliciaAronsson/RL_MasterThesis. The repository is structured as follows: The hedging environment is implemented in `env/hedging_env.py`, which defines the state space, action space, reward function and the GBM price dynamics. The three individual agents, DQN, DDPG and TD3, are implemented in `models/dqn_agent.py`, `models/ddpg_agent.py` and `models/td3_agent.py` respectively, while the hierarchical hybrid agent is found in `models/hybrid_agent.py`. The shared network architectures for the actor and critic are defined in `models/actor.py` and `models/critic.py`. Training procedures are located in the `train/` directory, and utility functions including the Black–Scholes pricing model, hedging cost evaluation and result visualisation are found in `utils/`. The main entry point for running all experiments is `main.py`, and all hyperparameters are found in `config.py`.

Confidentiality

The authors have no relevant confidentiality agreements to disclose.

Declarations

The authors have no relevant financial or non-financial interests to disclose.